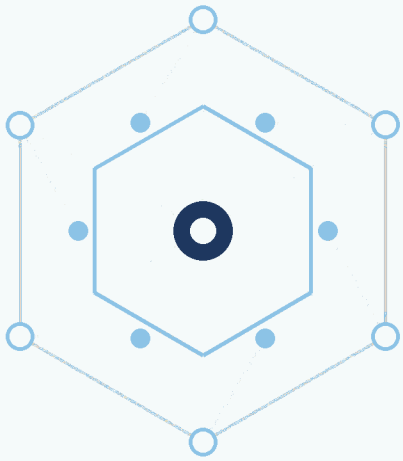
DUOCUC

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES



ASIGNATURA · ISY1102

Seguridad y calidad

en el desarrollo de software

Guías de estudio integradas · Unidades 1, 2 y 3

Resúmenes de las evaluaciones EA1, EA2 y EA3: de la calidad medible y el cumplimiento normativo chileno al diseño y certificación de pruebas, hasta el pipeline con controles de seguridad y el aseguramiento continuo.

EA1
UNIDAD 1

Calidad, seguridad y cumplimiento

Calidad ISO/IEC 25010 · Seguridad CIA · Cumplimiento · Error, defecto y falla · Verificación y validación · ISTQB · Marco normativo chileno · Plan de pruebas

EA2
UNIDAD 2

Diseño, ejecución y certificación de pruebas

Estrategia operativa · Docker · Caso de prueba · Caja negra y blanca · Trazabilidad · SonarQube · Bitácora e incidentes · Informe de certificación

EA3
UNIDAD 3

Automatización, DevSecOps y continuo

Pentesting ético · Pipeline CI/CD · SAST y DAST · Selenium · Carga y estrés con JMeter · Aseguramiento continuo

Presentación de la asignatura

Seguridad y calidad en el desarrollo de software estudia cómo planificar, diseñar, ejecutar y automatizar pruebas que demuestren —con evidencia— que un producto es útil, resistente a amenazas y conforme a las normas que le aplican. Calidad, seguridad y cumplimiento no son tres ramos separados: un software puede ser funcional e inseguro, seguro e ilegal, o legal e inutilizable. La asignatura pide equilibrio demostrable.

Esta guía unificada reúne en un solo documento el contenido esencial de las tres unidades, con índice general, numeración continua y un glosario consolidado al final. No incluye instrucciones de evaluación ni datos administrativos: solo el material para estudiar y aplicar.

Qué cubre

Unidad	Tema central	Pregunta que responde
1	Calidad, seguridad y cumplimiento	¿Qué se protege y mide, con qué normas, y cómo se traduce a un plan de pruebas?
2	Diseño, ejecución y certificación	¿Cómo se diseñan casos, se obtienen evidencias y se decide si el software se certifica?
3	Automatización y aseguramiento continuo	¿Cómo se automatizan pruebas y controles de seguridad en un pipeline CI/CD, y cómo se sostiene la calidad bajo carga?

Cómo se conectan las tres unidades

- La **Unidad 1** fija el vocabulario (calidad ISO/IEC 25010, seguridad CIA, cumplimiento, verificación frente a validación), el mapa normativo chileno y la estructura del plan de pruebas.
- La **Unidad 2** opera ese plan: técnicas de diseño, entorno reproducible (Docker), pruebas estáticas (SonarQube), bitácora de incidentes e informe de certificación.
- La **Unidad 3** escala el enfoque: pentesting ético, SAST/DAST en CI/CD, automatización funcional (Selenium), pruebas de estrés (JMeter) y mantenimiento continuo.

Hilo conductor. La Unidad 1 fija qué proteger y medir y con qué normas; la Unidad 2 convierte eso en casos, evidencias y certificación; la Unidad 3 automatiza y sostiene el ciclo en el pipeline y bajo carga.

Cómo estudiar. Lee cada bloque, detente en las *Ideas clave*, usa la chuleta antes de practicar, responde las preguntas sin mirar y recién después contraste. Los recuadros **Ojo** marcan trampas frecuentes; **Nota por confirmar** señala datos que conviene contrastar con la documentación oficial o la versión vigente de una norma.

Índice general

Presentación de la asignatura 2

Unidad 1. Calidad, seguridad y cumplimiento 4

1.1 Calidad, seguridad y cumplimiento como un solo objetivo 5
1.2 Normas que no se deben mezclar 7
1.3 Error, defecto, falla e incidente 8
1.4 Verificación, validación y principios ISTQB 9
1.5 Estrategia, nivel, tipo y técnica 10
1.6 Marco normativo chileno (mapa de estudio) 11
1.7 Plan de pruebas: de la norma al documento 12
1.8 Chuleta de la Unidad 1 13
1.9 Preguntas de autoevaluación 13
1.10 Ejercicios resueltos 14

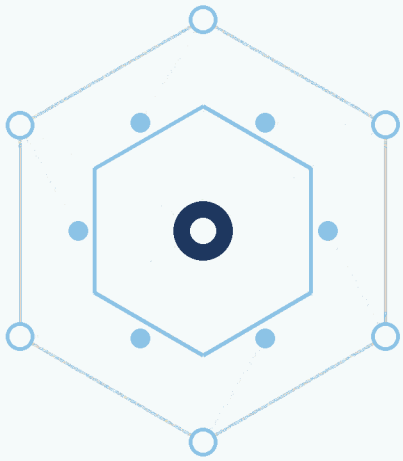
Unidad 2. Diseño, ejecución y certificación de pruebas 15

2.1 Del plan a la estrategia operativa 16
2.2 Entorno reproducible con Docker 20
2.3 Caso de prueba y plantilla 21
2.4 Técnicas de caja negra 23
2.5 Técnicas de caja blanca 24
2.6 Matriz de trazabilidad 26
2.7 Pruebas estáticas y SonarQube 27
2.8 Ejecución, bitácora e incidentes 30
2.9 Informe de certificación y ética 32
2.10 Catálogo de herramientas 37
2.11 Chuleta de la Unidad 2 39
2.12 Preguntas de autoevaluación 39
2.13 Ejercicios resueltos 40

Unidad 3. Automatización, DevSecOps y aseguramiento continuo 42

3.1 Pentesting: pruebas de penetración éticas 43
3.2 Pipeline CI/CD con controles de seguridad 45
3.3 Automatización de pruebas funcionales y mejora de scripts 47
3.4 Pruebas de carga, estrés y pico 48
3.5 Mantenimiento del aseguramiento continuo 49
3.6 Chuleta de la Unidad 3 50
3.7 Preguntas de autoevaluación 50
3.8 Ejercicios resueltos 51

Anexo. Glosario consolidado y referencia rápida 52



EVALUACIÓNEA1

Unidad 1. Calidad, seguridad y

cumplimiento

Cómo fijar el marco: qué es calidad del producto, qué es seguridad de la información en el software, qué exige el cumplimiento normativo en Chile, y cómo todo eso se convierte en un plan de pruebas trazable.

Punto de partida: el vocabulario y el mapa normativo. Lo que aquí se define (pilares, normas, verificación frente a validación) es lo que las unidades 2 y 3 operan y automatizan.

EA1

UNIDAD 1

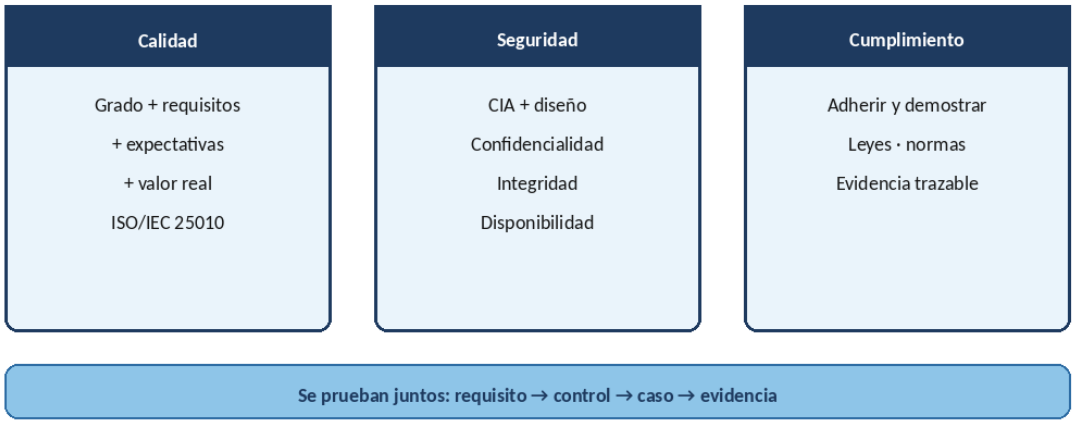
Contenidos de la unidad

1.1 Calidad, seguridad y cumplimiento como un solo objetivo
1.2 Normas que no se deben mezclar
1.3 Error, defecto, falla e incidente
1.4 Verificación, validación y principios ISTQB
1.5 Estrategia, nivel, tipo y técnica
1.6 Marco normativo chileno (mapa de estudio)
1.7 Plan de pruebas: de la norma al documento
1.8 Chuleta de la Unidad 1
1.9 Preguntas de autoevaluación
1.10 Ejercicios resueltos

Unidad 1. Calidad, seguridad y cumplimiento

La primera unidad construye el marco: qué es calidad del producto, qué es seguridad de la información en el software, qué exige el cumplimiento normativo en Chile, y cómo todo eso se convierte en un plan de pruebas trazable. Sin este vocabulario, las unidades siguientes se vuelven listas de herramientas sin criterio.

Cada término técnico se define la primera vez que aparece; cada bloque cierra con las ideas que conviene retener. Al final se incluyen una chuleta, quince preguntas de autoevaluación con respuesta y tres ejercicios resueltos.



Los tres pilares se prueban juntos

1.1 Calidad, seguridad y cumplimiento como un solo objetivo

1.1.1 Qué es calidad del software

La **calidad del software** es el **grado** en que un producto cumple **requisitos especificados** y **necesidades o expectativas** de quienes lo usan, aportando **valor real**. No es «cero defectos», ni solo velocidad de entrega, ni un eslogan de marketing. La palabra *grado* importa: dos productos pueden ser «de calidad» en medidas distintas según el contexto (un marcapasos y una app de notas no se juzgan igual).

La norma **ISO/IEC 25010** (modelo SQuaRE de calidad del producto) descompone esa calidad en características y subcaracterísticas. Eso permite diseñar tipos de prueba distintos: lo que mide *adecuación funcional* no es lo mismo que lo que mide *eficiencia de desempeño* o *seguridad*.

1.1.2 Qué es seguridad del software

La **seguridad del software** es la capacidad de proteger la información y el sistema frente a amenazas, de modo que personas o procesos accedan solo en el grado autorizado, y de resistir ataques. Se expresa con la tríada **CIA**:

Letra	Nombre	Pregunta	Ejemplo
C	Confidencialidad	¿Quién puede ver?	Historial médico solo con rol clínico

Letra	Nombre	Pregunta	Ejemplo
I	Integridad	¿Quién puede cambiar?	Nadie altera el monto de una transferencia
A	Disponibilidad	¿Cuándo está accesible?	El servicio no se cae el día de pago

Se complementa con **autenticidad** (probar que alguien es quien dice ser), **no repudio** (no poder negar una acción) y **responsabilidad** (*accountability*: poder demostrar quién hizo qué). El principio operativo es **Security by Design**: la seguridad no se «añade al final»; se diseña con el producto.

1.1.3 Qué es cumplimiento normativo

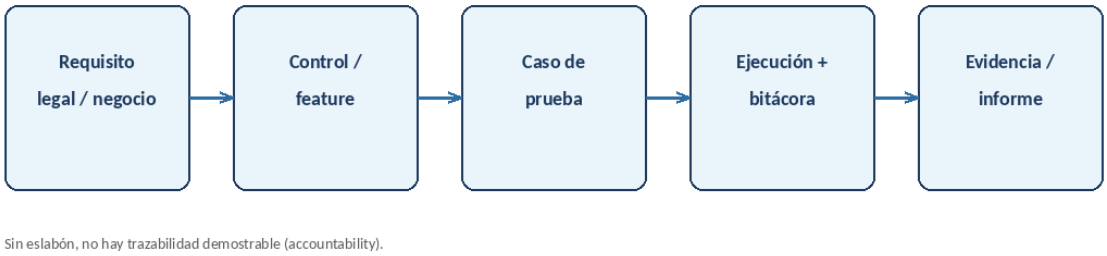
El **cumplimiento normativo** (*compliance*) es la adherencia del software y de los procesos que lo rodean a leyes, reglamentos, estándares técnicos y políticas internas. Cada exigencia normativa debe convertirse en un **requisito verificable** y luego en un **caso de prueba** o un **control revisable**. «Hay que cumplir la ley» sin caso ni evidencia es intención, no cobertura.

1.1.4 Por qué van juntos

Calidad, seguridad y cumplimiento no son tres ramos. Un software puede ser funcional e inseguro; seguro e ilegal; legal e inutilizable. La asignatura pide **equilibrio demostrable** (evidencias, no intenciones).

Pilar	En una frase	Si falla se ve como
Calidad	Grado + requisitos + expectativas + valor	Defectos, lentitud, mala UX, producto inútil
Seguridad	Proteger; CIA; desde el diseño	Fugas, alteraciones, caídas, ataques
Cumplimiento	Adherir y **demostrar** con evidencia	Multas, contratos perdidos, daño reputacional

Trazabilidad: requisito legal → código/feature → caso de prueba → evidencia. Sin caso, no hay cobertura. Eso es *accountability*: no basta cumplir, hay que **demostrarlo**.



Cadena requisito → evidencia

> **Ojo:** decir «hay que cumplir la ley y probar bien» sin nombrar norma, requisito, caso y evidencia es desempeño débil. El nivel que puntúa conecta definición + distinción (p. ej. 25010 frente a 27001) + caso + evidencia.

Ideas clave del bloque 1.1

- Calidad ≠ ausencia total de bugs; es grado y valor.
- Seguridad ≠ solo antivirus: es CIA + diseño.
- Cumplimiento exige trazabilidad requisito legal → control → evidencia.

1.2 Normas que no se deben mezclar

1.2.1 Mapa rápido de normas

Norma	Para qué sirve	No la uses para
ISO 9001	Gestión de calidad de la **organización** (procesos)	Atributos del producto software
ISO/IEC 25010	Calidad del **producto** (características y subcaracterísticas)	Montar un SGSI
ISO/IEC 27001	**SGSI**: seguridad de la información (riesgos + PDCA)	Sustituir el modelo 25010
ISO/IEC 12207	Procesos del ciclo de vida del software	Ser «la» norma de seguridad
ISO/IEC/IEEE 29119 (antes IEEE 829)	Procesos y documentación de pruebas	Reemplazar 27001 o 25010

Si una ley pide altos estándares de **seguridad de la información**, el marco de gestión adecuado es **27001**. Si pide especificar o medir calidad del **producto**, es **25010**.



Mapa de normas ISO

1.2.2 Características de calidad del producto (ISO/IEC 25010)

La versión 2011 popularizó ocho características; la **25010:2023** reorganiza el modelo (incluye, entre otras, capacidad de interacción donde antes se hablaba de usabilidad, flexibilidad donde antes portabilidad, y protección/*safety* como característica propia). En el material del curso suele trabajarse el esquema clásico de ocho; ambas lecturas sirven si no mezclas nombres.

Característica	Qué mira	Pruebas asociadas
Adecuación funcional	Completitud, corrección, pertinencia	Funcionales, reglas de negocio
Eficiencia de desempeño	Tiempo, recursos, capacidad	Carga, estrés, rendimiento
Compatibilidad	Coexistencia, interoperabilidad	Integración, compatibilidad
Usabilidad / interacción	Aprender, operar, errores de usuario, **accesibilidad**	Usabilidad, accesibilidad
Fiabilidad	Madurez, disponibilidad, tolerancia, recuperación	Recuperación, disponibilidad
Seguridad	CIA + no repudio, responsabilidad, autenticidad	Authn/authz, entradas, config
Mantenibilidad	Modularidad, reuso, análisis, cambio, **capacidad de ser probado**	Análisis estático, revisión
Portabilidad / flexibilidad	Adaptabilidad, instalabilidad, reemplazo	Compatibilidad, despliegue

> **Ojo:** no digas «probabilidad» al hablar de mantenibilidad: di **capacidad para ser probado**. Lectores de pantalla = **accesibilidad** (dentro de usabilidad/interacción), no un tipo suelto sin ancla.

> **Nota por confirmar:** al citar subcaracterísticas o el conteo exacto de características, verifica si tu fuente usa 25010:2011 o 25010:2023; los nombres cambiaron en varios puntos.

Ideas clave del bloque 1.2

- 9001 ≠ 25010 ≠ 27001: organización, producto y SGSI no se intercambian.
- Cada característica de 25010 orienta tipos de prueba distintos.

1.3 Error, defecto, falla e incidente

1.3.1 Cadena causal

En pruebas conviene no mezclar cuatro palabras:

Término	Dónde vive	Ejemplo
Error	En la mente/acción humana	Malentender la regla de negocio
Defecto (*bug / fault*)	En el artefacto	Condición mal codificada
Falla (*failure*)	En la ejecución	El sistema calcula mal el total
Incidente	En la operación/gestión	Ticket: «no puedo pagar»

Un error puede introducir un defecto; el defecto puede permanecer latente; al ejecutarse el camino afectado aparece la falla; si alguien la reporta o impacta el servicio, se gestiona como incidente.

1.3.2 Por qué importa la distinción

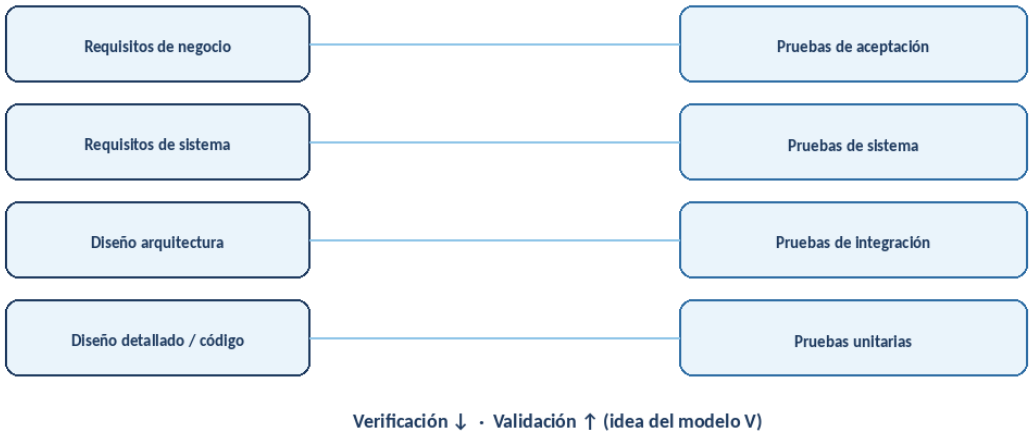
Si etiquetas todo como «bug», no sabes si falló el requisito, el código, el entorno o la operación. La bitácora y el informe de certificación necesitan esa precisión: un incidente puede deberse a un defecto nuevo, a uno conocido, a un problema de datos o a un malentendido del usuario.

> **Ojo:** no todo incidente implica un defecto de producto (puede ser capacitación, dato externo, infraestructura). Investiga antes de abrir un defecto.

Ideas clave del bloque 1.3

- Error → defecto → falla → incidente no son sinónimos.
- La gestión del incidente incluye clasificación y evidencia, no solo «arreglar algo».

1.4 Verificación, validación y principios ISTQB



Modelo V: verificación y validación

1.4.1 Verificación frente a validación

	Verificación	Validación
Pregunta	¿Lo construimos **bien**?	¿Construimos lo **correcto**?
Contra qué	Especificaciones, diseño, estándares	Necesidad real / uso / negocio
Ejemplos	Revisión de código, pruebas unitarias contra diseño	Prueba de aceptación con usuario

En el **modelo V**, el lado izquierdo desciende especificando; el derecho asciende probando. Cada nivel de prueba se ancla a un nivel de especificación.

1.4.2 Principios ISTQB (lectura de estudio)

1. Las pruebas demuestran presencia de defectos, no su ausencia absoluta.
2. El testing exhaustivo es imposible: se prioriza.
3. Probar temprano es más barato.
4. Los defectos se agrupan (cluster).
5. Paradoja del pesticida: mismas pruebas dejan de encontrar cosas nuevas.

6. El testing depende del contexto.
7. Ausencia de errores ≠ producto útil (falta validación / valor).

> **Nota por confirmar:** la redacción exacta de los principios varía levemente entre syllabus ISTQB; retén la idea, no un número de versión de memoria.

Ideas clave del bloque 1.4

- Verificación ≠ validación.
- Priorizar por riesgo: no se puede probar «todo».

1.5 Estrategia, nivel, tipo y técnica



Niveles de prueba

1.5.1 Cuatro palabras de planificación

Concepto	Pregunta que responde
Estrategia	¿Cuál es el enfoque general y las prioridades (riesgos, automatización, entornos)?
Nivel	¿En qué momento del ciclo / capa del V? (unidad, integración, sistema, aceptación)
Tipo	¿Qué aspecto de calidad? (funcional, rendimiento, seguridad, usabilidad…)
Técnica	¿Cómo derivo los casos? (equivalencia, límites, tabla de decisión, cobertura…)

1.5.2 Niveles (resumen)

- **Componente/unidad:** cada pieza aislada.
- **Integración:** interfaces y colaboración.
- **Sistema:** producto integrado contra requisitos.

- **Aceptación:** valor para usuario/negocio (validación).

Mezclar nivel y tipo es un error típico: «prueba de sistema» no dice si miras funcionalidad o seguridad; «prueba de seguridad» puede hacerse en varios niveles.

Ideas clave del bloque 1.5

- Estrategia orienta; nivel, tipo y técnica aterrizan el plan.
- Un mismo tipo (p. ej. seguridad) puede aparecer en varios niveles.

1.6 Marco normativo chileno (mapa de estudio)

1.6.1 Para qué sirve el mapa

No memorices artículos de memoria: aprende **qué ley anclar** según el problema (datos personales, ciberseguridad, delitos, propiedad intelectual) y cómo se traduce a requisito verificable.

Ley / norma	Idea en una línea	Ancla de pruebas
21.663 Ciberseguridad (2024)	ANCI, operadores, requisitos mínimos, reporte, Security by Design	Controles de seguridad, logging, respuesta
21.719 Datos personales (2024)	Enfoque tipo GDPR; principios; responsabilidad proactiva	Consentimiento, minimización, accesos, plazos
19.628 Datos (1999)	Régimen previo aún citado en material legacy	Cifrado, accesos, trazabilidad
21.459 Delitos informáticos	Tipifica ataques/acceso ilícito	Autorización de pentest; no confundir con OSS
17.336 Propiedad intelectual	Licencias y derechos de autor	Inventario de dependencias / licencias
21.180 Transformación digital del Estado	Interoperabilidad y digitalización (sector público)	Interfaces, accesibilidad según alcance

1.6.2 Siete principios (21.719) — estudio

1. Licitud, lealtad y transparencia
2. Limitación de la finalidad
3. Minimización
4. Exactitud
5. Limitación del plazo de conservación
6. Integridad y confidencialidad
7. Responsabilidad proactiva (**demostrar** cumplimiento)

> **Ojo:** la ley de datos aplica a **datos personales**, no solo a tarjetas bancarias. Salud, correo y GPS también cuentan.

> **Nota por confirmar:** montos de sanción en UTM y matices de vigencia/reglamentación: verifica el texto oficial vigente antes de citar cifras en un informe formal.

Ideas clave del bloque 1.6

- Elige la ley según el riesgo (datos vs ciber vs delito vs PI).
- Principio 7: cumplir no basta; hay que poder demostrarlo.

1.7 Plan de pruebas: de la norma al documento

1.7.1 Qué es

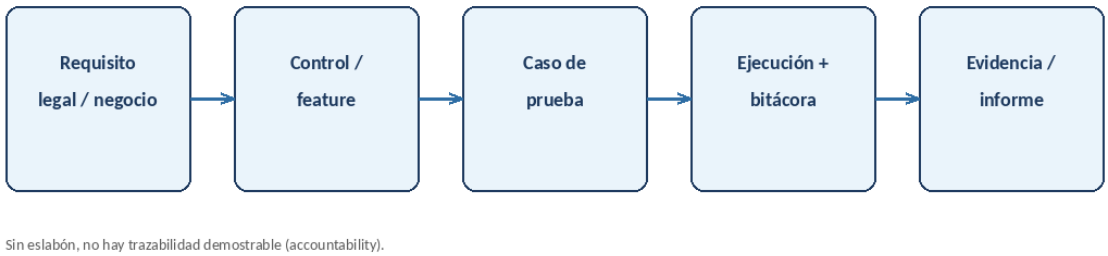
El **plan de pruebas** es el documento (o conjunto) que responde: qué se probará y qué no, con qué enfoque, en qué entornos, con qué criterios de entrada/salida, quién hace qué, y qué evidencias se guardarán. Estándares de referencia de estudio: familia **ISO/IEC/IEEE 29119** e ideas heredadas de **IEEE 829**.

1.7.2 Contenidos mínimos útiles

Bloque	Contenido
Identificación	Proyecto, versión, referencias al ERS/normas
Alcance	Incluye / excluye
Riesgos	Qué prioriza la estrategia
Enfoque	Niveles, tipos, técnicas, automatización
Criterios	Entrada, salida, suspensión, reanudación
Entorno	Datos, herramientas, Docker/labs
Roles	Quién diseña, ejecuta, aprueba
Entregables	Casos, bitácoras, informes, evidencias

1.7.3 De la norma al caso

Norma/ley → requisito del ERS → control en el producto → caso de prueba → evidencia. El plan declara esa cadena; la Unidad 2 la opera; la Unidad 3 la automatiza.



Cadena requisito → evidencia

Ideas clave del bloque 1.7

- Plan sin criterios de salida no se puede cerrar.
- Plan sin riesgos prioriza mal el esfuerzo.

1.8 Chuleta de la Unidad 1

Tema	Recuerda
Calidad	Grado + requisitos + expectativas + valor (25010)
Seguridad	CIA + Security by Design
Cumplimiento	Adherir **y demostrar** (trazabilidad)
9001 / 25010 / 27001	Organización / producto / SGSI
Error → defecto → falla → incidente	Humano → en el artefacto → en ejecución → en operación
Verificación vs validación	¿Bien hecho? vs ¿hecho lo correcto?
Principios ISTQB	Agotar es imposible; defectos se agrupan; paradoja del pesticida; contexto
Estrategia / nivel / tipo / técnica	Enfoque · dónde en el V · qué calidad · cómo diseñar casos
Leyes Chile (mapa)	21.663 ciber; 21.719 datos; 19.628; 21.459 delitos; 17.336 PI
Plan de pruebas	Alcance, riesgos, niveles/tipos, entrada/salida, evidencias

1.9 Preguntas de autoevaluación

1. **¿Qué significa que la calidad sea un «grado»?** Que no es binaria: se mide en qué medida el producto cumple requisitos y expectativas con valor real, según el contexto.
2. **¿Qué pregunta responde cada letra de CIA?** Confidencialidad: quién puede ver; integridad: quién puede cambiar; disponibilidad: cuándo está accesible.
3. **¿Qué distingue cumplimiento de «intención de cumplir»?** La evidencia trazable: requisito → control → caso → registro. Sin caso, no hay cobertura demostrable.
4. **¿Cuándo usarías 27001 y cuándo 25010?** 27001 para gestionar seguridad de la información (SGSI); 25010 para especificar/medir calidad del producto.
5. **¿ISO 9001 sirve para listar atributos de un producto software?** No: 9001 mira la gestión de calidad de la organización, no el modelo de calidad del producto.
6. **¿Qué es un defecto frente a un error?** El error es humano (malentendido, tipo); el defecto es la huella en el artefacto (código, diseño, requisito).
7. **¿Cuándo un defecto se vuelve falla?** Cuando se ejecuta el artefacto y el comportamiento se desvía de lo esperado.
8. **¿Verificación o validación: revisar que el módulo cumple su especificación de diseño?** Verificación («¿lo construimos bien?»).
9. **¿Verificación o validación: una prueba de aceptación con el usuario final?** Validación («¿construimos lo correcto?»).
10. **¿Por qué «agotar las pruebas» es imposible?** El espacio de entradas, estados y caminos crece combinatoriamente; se prioriza por riesgo y valor.
11. **¿Qué es la paradoja del pesticida en testing?** Repetir siempre las mismas pruebas deja de encontrar defectos nuevos; hay que renovar casos y datos.
12. **¿Qué diferencia nivel, tipo y técnica?** Nivel: cuándo/dónde en el ciclo (unidad…aceptación). Tipo: qué aspecto de calidad. Técnica: cómo derivar casos.

13. **¿Qué exige la Ley 21.719 en una frase de estudio?** Proteger datos personales con principios (minimización, finalidad, responsabilidad proactiva, etc.) alineados a un enfoque tipo GDPR.
14. **¿Qué aporta la Ley 21.663 al diseño de software?** Marco de ciberseguridad (ANCI, operadores, requisitos mínimos, reporte, Security by Design en infraestructuras relevantes).
15. **¿Qué debe contener, como mínimo, un plan de pruebas?** Alcance y objetivos; riesgos; niveles/tipos/técnicas; criterios de entrada/salida; entorno; roles; evidencias y reportes.

1.10 Ejercicios resueltos

1.10.1 Ejercicio 1: ¿qué norma anclar?

Un enunciado pide «altos estándares de seguridad de la información» y, en otro párrafo, «medir la usabilidad y el desempeño del producto». ¿Qué norma anclas en cada frase y por qué?

Solución. Seguridad de la información → **ISO/IEC 27001** (SGSI, riesgos, controles). Usabilidad y desempeño del producto → **ISO/IEC 25010** (características de calidad del producto). Mezclarlas (pedir 27001 «para usabilidad» o 25010 «como SGSI») es el error típico del bloque 1.2.

1.10.2 Ejercicio 2: clasificar el hallazgo

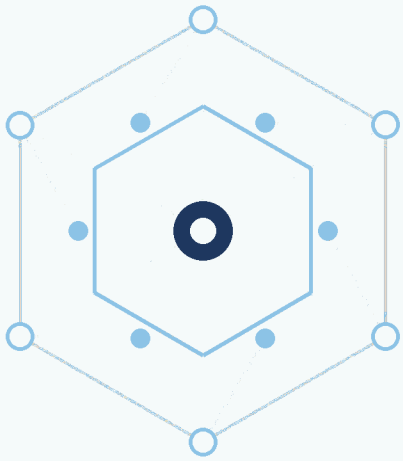
Durante una prueba, el analista escribe mal la contraseña de prueba en el script (1). El script deja hardcodeada una clave en el repositorio (2). En ejecución, el login acepta la clave hardcodeada y deja entrar sin política de expiración (3). En producción, un usuario reporta acceso indebido (4). Etiqueta error, defecto, falla e incidente.

Solución. (1) **Error** humano. (2) **Defecto** en el artefacto (código/config). (3) **Falla** en ejecución (comportamiento incorrecto observable). (4) **Incidente** en operación (evento reportado que requiere gestión).

1.10.3 Ejercicio 3: de la ley al caso

La app de salud pide consentimiento para datos clínicos. Traduce a requisito, control, caso y evidencia.

Solución. Requisito: «No se envían datos clínicos sin consentimiento registrado». Control: checkbox obligatorio + registro de auditoría con marca temporal. Caso: intentar continuar sin aceptar → el sistema bloquea y muestra mensaje; con aceptación → queda log. Evidencia: captura de pantalla + extracto de log con id de usuario y timestamp. Sin el caso y la evidencia, el «cumplimos la 21.719» no se demuestra.



EVALUACIÓNEA2

Unidad 2. Diseño, ejecución y

certificación de pruebas

Cómo operar el plan: estrategia en laboratorio, entorno reproducible con Docker, diseño de casos (caja negra y blanca), trazabilidad, pruebas estáticas con SonarQube, bitácora de incidentes e informe de certificación con ética profesional.

Viene de la Unidad 1: con el plan y el marco normativo claros, aquí se diseñan casos, se obtienen evidencias y se decide si el software se certifica. Prepara la Unidad 3, que automatiza y sostiene ese ciclo.

EA2

UNIDAD 2

Contenidos de la unidad

2.1 Del plan a la estrategia operativa
2.2 Entorno reproducible con Docker
2.3 Caso de prueba y plantilla
2.4 Técnicas de caja negra
2.5 Técnicas de caja blanca
2.6 Matriz de trazabilidad
2.7 Pruebas estáticas y SonarQube
2.8 Ejecución, bitácora e incidentes
2.9 Informe de certificación y ética
2.10 Catálogo de herramientas
2.11 Chuleta de la Unidad 2
2.12 Preguntas de autoevaluación
2.13 Ejercicios resueltos

Unidad 2. Diseño, ejecución y certificación de pruebas

La Unidad 2 opera el marco de la Unidad 1: convierte el plan en casos ejecutables, en un entorno reproducible, con evidencias y un informe de certificación. Cada término técnico se define la primera vez que aparece; cada bloque cierra con las ideas que conviene retener. Al final hay chuleta, quince preguntas con respuesta y tres ejercicios resueltos.

Los recuadros **Ojo** advierten trampas frecuentes del laboratorio. Los recuadros **Nota por confirmar** señalan datos que conviene validar con la documentación oficial o la versión vigente de una norma.



Niveles de prueba

2.1 Del plan a la estrategia operativa

3.1 Qué es un plan de pruebas

Según 2.1.1, un plan de pruebas es **un marco estratégico que organiza, estructura y guía todas las actividades de validación del producto**. Su propósito es asegurar la calidad, detectar defectos en etapas tempranas y validar la seguridad antes del lanzamiento. Su valor práctico es triple: reduce riesgos, mejora la comunicación entre equipos y da una base objetiva para decidir si el software se entrega.

Un plan no es una lista de casos. Los casos son una consecuencia del plan.

3.2 Elementos del plan

El material reúne los elementos en cuatro bloques (2.1.1: alcance y objetivos, estrategia, recursos y roles, criterios de aceptación) La lista completa que conviene cubrir es:

#	Elemento	Qué debe contener	Error típico
1	**Introducción**	Contexto del sistema, propósito del plan, estructura del documento	Describir la empresa y no el software

#	Elemento	Qué debe contener	Error típico
2	**Alcance**	Qué módulos y requisitos se prueban y **qué queda fuera y por qué**	Omitir las exclusiones
3	**Objetivos**	Objetivos medibles (general y específicos)	Objetivos vagos: "asegurar la calidad"
4	**Criterios de calidad**	Usabilidad, rendimiento, estabilidad, confiabilidad, mantenibilidad, seguridad y normativa	Listar atributos sin umbrales
5	**Estrategia**	Enfoque (manual, automatizado, mixto), etapas, priorización, criterios de entrada y salida	Confundir estrategia con lista de herramientas
6	**Tipos de prueba**	Cada tipo con su justificación y los requisitos que cubre	Nombrar tipos sin decir para qué
7	**Criterios de aceptación**	Condiciones transversales para aprobar casos y para cerrar el ciclo	Criterios no medibles
8	**Recursos**	Humanos (roles), técnicos (software, versiones, navegadores) y entorno (desarrollo, QA)	No declarar versiones
9	**Herramientas**	Cada herramienta con el tipo de prueba y el requisito que respalda	Herramientas que luego no se usan
10	**Cronograma**	Fases y fechas, priorizadas por riesgo	Cronograma sin relación con los riesgos
11	**Riesgos y mitigación**	Retrasos, falta de cobertura, fallos críticos, dependencias externas	Solo riesgos del producto, no del proyecto de pruebas
12	**Trazabilidad**	Matriz requisito ↔ caso ↔ riesgo ↔ resultado	Matriz parcial

Ejemplo de objetivo bien escrito (2.1.1): "Verificar que el sistema de autenticación implementa cifrado seguro, maneja correctamente las sesiones y previene accesos no autorizados". Es específico, se puede comprobar y apunta a requisitos concretos.

3.3 Criterios de entrada, salida y suspensión

Criterio	Pregunta	Ejemplo
Entrada	¿Cuándo se puede empezar a ejecutar?	Contenedores arriba; datos de prueba cargados; casos revisados; smoke test aprobado
Suspensión	¿Cuándo se detiene la ejecución?	El smoke test falla; el entorno se cae; un defecto bloquea el flujo principal
Reanudación	¿Qué se necesita para seguir?	Corrección desplegada y verificada

Criterio	Pregunta	Ejemplo
Salida	¿Cuándo se da por terminado el ciclo?	100 % de los casos de prioridad alta ejecutados; 0 defectos críticos abiertos; cobertura de requisitos sobre el umbral

3.4 Estrategia basada en riesgos

El riesgo combina la **probabilidad** de que algo falle con el **impacto** si falla. La priorización resultante define qué se diseña con más detalle, qué se ejecuta primero y qué recibe revisión de código.

	Impacto bajo	Impacto medio	Impacto alto
Probabilidad alta	Media	Alta	**Crítica**
Probabilidad media	Baja	Media	Alta
Probabilidad baja	Baja	Baja	Media

Criterios que el material entrega para estimar el riesgo:

- **Exposición:** primero los módulos expuestos directamente a internet, como API públicas, formularios de autenticación y pasarelas de pago (2.1.1).
- **Complejidad:** módulos con complejidad ciclomática V(G) > 20 requieren más profundidad, revisión de código, pruebas negativas y de estrés, y prioridad en el cronograma (2.1.2).
- **Sensibilidad de los datos:** credenciales, datos personales y datos sensibles (salud, finanzas).
- **Impacto en el negocio:** funcionalidades centrales para el usuario.

3.6 Recursos, roles e independencia

- **Independencia del testing:** quien prueba no debería ser quien desarrolló; aporta objetividad y mirada crítica.
- **Roles especializados:** líder de QA, testers funcionales, especialistas en automatización y especialistas en seguridad.
- **Entornos:** aislados, reproducibles y con escáneres actualizados.
- **Comunicación:** documentar → revisar con los interesados → comunicar avances → aprobar formalmente antes de ejecutar (ciclo de 2.1.1).

Desafíos comunes que el material identifica y su solución:

Desafío	Solución propuesta
Pruebas que empiezan tarde	*Shift-left*: integrar pruebas desde el diseño
Falta de especialistas	Capacitación continua, certificaciones y mentorías internas
Desalineación entre equipos	Ceremonias regulares, documentación compartida y objetivos comunes

Flujo de trabajo de seis pasos (2.1.1): (1) definir alcance y objetivos medibles; (2) diseñar una estrategia basada en riesgos; (3) planificar recursos y cronograma; (4) elaborar casos funcionales y de seguridad trazables; (5) integrar automatización y KPI de cobertura, eficacia y eficiencia; (6) documentar y revisar de forma continua.

3.7 Automatización y seguridad continua

- **Automatización de regresión:** casos repetitivos que se ejecutan en cada cambio de código.
- **Seguridad continua:** SAST y DAST integrados al *pipeline* de integración y despliegue continuo (CI/CD). Este tema se profundiza en la EA3.
- El material presenta cifras de beneficio (70 % menos tiempo de regresión, 3 veces más casos con la misma inversión). Son ilustrativas; en un informe es mejor medir el propio ahorro.

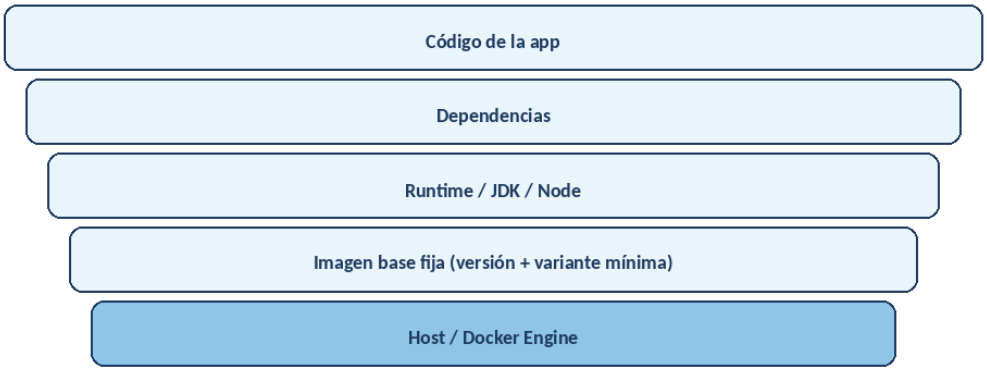
SAST frente a DAST

	SAST (estático)	DAST (dinámico)
¿Requiere ejecutar la app?	No	Sí
¿Qué analiza?	Código fuente	Aplicación desplegada, desde afuera
¿Cuándo se usa?	Desde que existe código	Cuando hay un entorno funcionando
Fortaleza	Indica archivo y línea	Confirma que la falla es explotable
Debilidad	Falsos positivos; no ve la configuración del despliegue	No ve el código; depende de la cobertura del rastreo
Herramientas del curso	SonarQube	OWASP ZAP, Burp Suite, Nikto, SQLMap

3.8 Marcos y estándares de referencia del plan

Marco	Para qué se usa en el plan
IEEE 829	Plantillas de documentación de pruebas: plan, diseño, casos, procedimientos, registro de incidentes e informe resumen. Fue reemplazado por la serie ISO/IEC/IEEE 29119, pero el curso usa 829 como referencia de plantilla.
ISTQB	Vocabulario, principios y proceso de pruebas.
OWASP WSTG	Guía de escenarios de pruebas de seguridad web (autenticación, autorización, sesiones, validación de entradas, manejo de errores).
OWASP Top 10	Lista de las categorías de riesgo más críticas en aplicaciones web; la edición vigente es la de 2025. Sirve para clasificar hallazgos.
NIST Cybersecurity Framework	Marco de gestión del riesgo de ciberseguridad a nivel organizacional. La versión 2.0 (2024) tiene seis funciones: Gobernar, Identificar, Proteger, Detectar, Responder y Recuperar.

2.2 Entorno reproducible con Docker



Capas de una imagen Docker

3.9 Docker en el plan de pruebas (2.1.1 y guía 2.1.3)

Por qué Docker. Una **imagen** es una plantilla inmutable que encapsula sistema operativo, bibliotecas, dependencias y configuración. Un **contenedor** es una instancia en ejecución de esa imagen. El flujo es `Dockerfile → imagen → contenedor`. Para pruebas esto significa entornos replicables, versiones controladas, aislamiento y la posibilidad de volver a un estado anterior.

Buenas prácticas de seguridad en imágenes (que también pueden ser hallazgos):

Práctica	Qué revisar
Imágenes base oficiales y mínimas (por ejemplo, Alpine)	Menor superficie de ataque
Imágenes endurecidas (*hardened*)	Menos vulnerabilidades conocidas desde el inicio
Minimizar capas y dependencias	Cada componente extra es un posible vector
Privilegios mínimos	El proceso no debería correr como `root` dentro del contenedor
Aislamiento	Espacios de nombres y restricción de capacidades de Linux
Versionado y etiquetado	Versionado semántico y etiquetas descriptivas con fecha y cambios
Secretos fuera de la imagen	Variables sensibles en un `.env` no versionado o en un gestor de secretos
Automatización	Construcción y despliegue de imágenes en CI/CD para reducir errores humanos

Procedimiento de la guía 2.1.3 (individual, 2 horas, IL2.1):

1. Tener Docker instalado (sitio oficial) y abrir el proyecto en Visual Studio Code.
2. Descomprimir `CodigoFuenteA.zip` o `CodigoFuenteB.zip`, según el caso asignado.
3. Reconocer la estructura:

- `BACKEND`: código de la API;
- `FRONTEND`: código del cliente;

- `.env`: variables de entorno que usa Docker para construir las imágenes;
- `docker-compose.yml`: define cómo se construyen las imágenes, sus variables y la orquestación entre contenedores;
- `README.md`: información del proyecto.

4. Iniciar Docker, abrir una terminal en la raíz y ejecutar `docker-compose up -d`.
5. Verificar en Docker que existan **tres contenedores** activos (base de datos, API y cliente) con el nombre del caso.
6. Abrir el cliente y la documentación de la API (Swagger en `/docs`). Los puertos se leen en el contenedor o en `docker-compose.yml`:

Componente	Caso A (Atlas)	Caso B (Cuidarte+)
Cliente (frontend)	`http://localhost:3350`	`http://localhost:3333`
API y Swagger	`http://localhost:4450/docs`	`http://localhost:4444/docs`
PostgreSQL	`localhost:15432`	`localhost:15442`

Sin un entorno estable no hay ejecución confiable. Por eso el levantamiento del entorno es un **criterio de entrada** del plan, y su descripción (versiones, puertos, datos semilla) forma parte de la sección de recursos del informe.

> **Repaso 3.** Nombra cinco elementos del plan y un error típico de cada uno. ¿Qué módulo priorizarías primero en un sistema con login, reportes y un formulario de contacto, y por qué? ¿Qué diferencia hay entre una imagen y un contenedor?

2.3 Caso de prueba y plantilla

4.1 Definiciones

- **Caso de prueba (IEEE 610):** conjunto estructurado de entradas, condiciones de ejecución y resultados esperados, desarrollado para validar un requisito específico del sistema.
- **Escenario de prueba:** el **qué**; una situación de alto nivel. Ejemplo (2.2.2): "Validar el proceso de recuperación de contraseña".
- **Caso de prueba:** el **cómo**; los pasos detallados. Del escenario anterior salen al menos tres casos: recuperación con correo válido, intento con correo no registrado y expiración del enlace (este último es de seguridad).

Un buen caso es **específico, medible, reproducible y trazable** (2.1.1). Cualquier integrante del equipo debería poder ejecutarlo y obtener el mismo resultado. Además, los casos bien diseñados sirven de guía uniforme de ejecución, mejoran la colaboración entre desarrollo, QA y negocio, y facilitan la automatización (2.2.1).

4.2 Campos del caso de prueba

La unidad trabaja con tres listas de campos que conviene unificar en una sola plantilla:

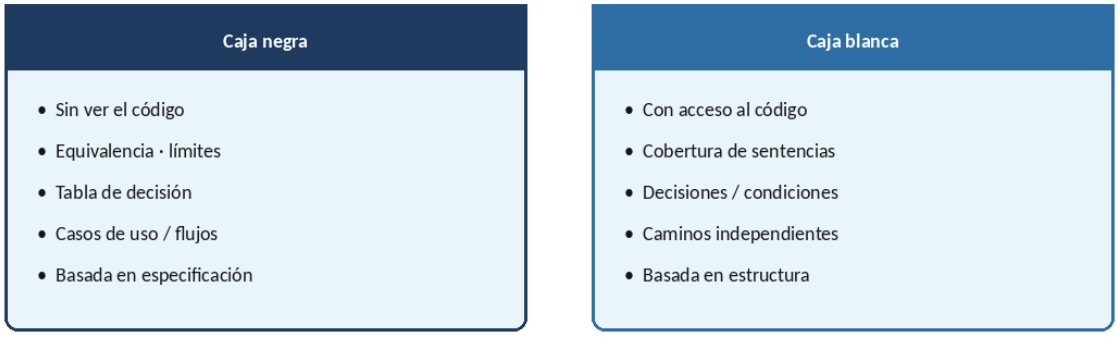
Campo	2.2.2 (IEEE 829)	Formato informe de certificación	Pauta informe de certificación	Para qué sirve
ID del caso	✔	✔	✔	Identificación única (CP-01, CP-SEG-001)
Nombre		✔		Lectura rápida en la bitácora
Descripción / objetivo		✔	✔	Qué se busca comprobar
Requisito asociado (código ERS)		✔	✔	Trazabilidad
Tipo de prueba			✔	Clasificación (IE3)
Prioridad (por riesgo)	✔			Orden de ejecución
Precondiciones	✔	✔		Estado inicial necesario
Pasos de ejecución	✔	✔		Acciones atómicas numeradas
Datos de prueba / de entrada	✔	✔	✔	Valores exactos usados
Resultado esperado	✔	✔	✔	Comportamiento correcto
Criterio de aceptación		✔	✔	Condición objetiva de aprobación
Resultado obtenido	✔			Se completa al ejecutar
Estado	✔			Aprobado / Falla / Bloqueado
Postcondiciones	(2.2.1)			Estado en que queda el sistema

Ejemplo completo de caso de seguridad (adaptado del ejemplo de 2.1.1, válido para cualquier sistema con login):

Campo	Contenido
ID	CP-SEG-001
Nombre	Rechazo de inyección SQL en el inicio de sesión
Requisito	Requisito de autenticación y de validación de entradas del ERS
Tipo	Seguridad (validación de entradas)
Prioridad	Alta
Precondiciones	Servicio activo; existe al menos un usuario registrado
Pasos	1. Abrir la pantalla de login. 2. Ingresar `' OR '1'='1' --` en usuario y cualquier valor en contraseña. 3. Enviar. 4. Revisar la respuesta y los logs
Datos	Usuario: `' OR '1'='1' --`; contraseña: `x`
Resultado esperado	Acceso denegado; mensaje genérico; sin detalles de base de datos en la respuesta; evento registrado en el log de seguridad

Campo	Contenido
Criterio de aceptación	HTTP 400 o 401, ninguna sesión creada y ningún texto de error SQL visible

2.4 Técnicas de caja negra



Caja negra frente a caja blanca

4.3 Técnicas de caja negra

Se centran en entradas y salidas, sin conocer la implementación. Validan el sistema desde la perspectiva del usuario.

a) Partición de equivalencia. Divide el dominio de entrada en clases donde el sistema debería comportarse igual; basta un valor representativo por clase.

Pasos: (1) identificar el dominio de entrada; (2) dividir en clases equivalentes, válidas e inválidas; (3) elegir un valor por clase.

Ejemplo (2.2.1): campo que acepta números de 1 a 10.

ID	Valor	Clase	Resultado esperado
TC-01	5	Válida (1–10)	Aceptado
TC-02	-3	Inválida inferior	Rechazado con mensaje
TC-03	15	Inválida superior	Rechazado con mensaje
TC-04	"abc"	Inválida de tipo	Rechazado con mensaje

Si el dominio tiene dos rangos válidos (1–10 y 20–30), aparecen clases adicionales; el material lo usa para mostrar que las clases dependen del requisito y no de una regla fija.

b) Análisis de valores límite (BVA). Prueba los bordes de cada partición, porque ahí aparecen los errores de comparación (usar < en lugar de ≤). Para cada límite se prueba el valor anterior, el límite y el siguiente.

Caso	Valor	Descripción	Esperado
1	0	Justo antes del límite inferior	Rechazado

Caso	Valor	Descripción	Esperado
2	1	Límite inferior	Aceptado
3	2	Dentro del rango	Aceptado
4	9	Cerca del límite superior	Aceptado
5	10	Límite superior	Aceptado
6	11	Justo después del límite superior	Rechazado

Aplicación típica: largo mínimo de contraseña. Si el requisito pide 12 caracteres o más, se prueban 11 (rechazo), 12 (aceptación) y 13 (aceptación). Para un nombre de 1 a 100 caracteres (ejemplo de 2.2.2): 0, 1, 2, 99, 100 y 101.

Partición y valores límite se usan juntas: la partición define las clases y los valores límite prueban sus fronteras.

c) Tablas de decisión. Útiles cuando el resultado depende de combinaciones de condiciones. Componentes: **condiciones**, **combinaciones (reglas)** y **acciones**. Con *n* condiciones binarias hay 2^n reglas. Ventajas: muestran todas las combinaciones, evitan omisiones, revelan inconsistencias en la lógica de negocio y son legibles para personas no técnicas.

Ejemplo (2.2.1): formulario con nombre y correo.

	R1	R2	R3	R4
C1 Nombre completo	No	No	Sí	Sí
C2 Correo válido	No	Sí	No	Sí
Botón habilitado	No	No	No	Sí
Permitir envío	No	No	No	Sí
Mensaje	Ambos campos	Falta nombre	Correo inválido	—

Aplicación típica en seguridad: rol del usuario × recurso solicitado × propiedad del recurso → permitir o denegar.

2.5 Técnicas de caja blanca

4.4 Técnicas de caja blanca

Requieren acceso al código, a los flujos de control y a la arquitectura. Validan que la lógica interna sea correcta y que no queden caminos sin ejecutar.

Técnica	Qué asegura	Ejemplo
Cobertura de sentencias	Cada línea se ejecuta al menos una vez	Mínimo aceptable, no suficiente
Cobertura de decisiones (ramas)	Cada `if` se evalúa verdadero y falso	El `if (!user)` del login con usuario existente y con usuario inexistente
Cobertura de condiciones	Cada condición simple dentro de una expresión compuesta toma ambos valores	`if (token && rol === 'admin')`: token válido sin rol, rol sin token, ambos

Técnica	Qué asegura	Ejemplo
Caminos básicos	Cada camino linealmente independiente se ejecuta al menos una vez	Número de caminos = V(G)
Pruebas de bucles	Ciclos `for`, `while` y `do-while` con distintas iteraciones	Ver tabla siguiente
Flujo de datos	Uso correcto de variables: definición–uso	Variable usada sin definir; definida y redefinida sin uso; dato sensible que llega a un log

Complejidad ciclomática (McCabe, 1976). Mide el número de caminos linealmente independientes y, por tanto, el **mínimo de casos** para cubrir todos los caminos básicos.

- Fórmula: **V(G) = E − N + 2P** (E = aristas, N = nodos, P = componentes conectados; para una sola función, P = 1).
- Atajo práctico: V(G) = número de decisiones simples + 1.

Ejemplo: un grafo con 9 aristas, 7 nodos y 1 componente → V(G) = 9 − 7 + 2 = **4** → al menos 4 casos. La pregunta de la pausa de 2.2.1 lo confirma: con V(G) = 5 se necesitan como mínimo 5 casos.

V(G)	Interpretación (2.2.1)	Acción
1–10	Código simple, fácil de probar	Pruebas normales
11–20	Complejidad moderada	Más casos, revisión
21–50	Código complejo	Refactorizar; prioridad alta en el plan
> 50	Muy complejo, alto riesgo	Refactorizar antes de certificar

Pruebas de bucles

Tipo de bucle	Cómo se prueba
Simple (máximo *n*)	0 iteraciones, 1, 2, un valor típico *m* < *n*, *n*−1, *n* y *n*+1
Anidado	Desde el más interno hacia afuera, dejando los demás en valores mínimos
Concatenado	Si son independientes, como simples; si dependen entre sí, como anidados

4.5 Técnicas híbridas

- **Basadas en riesgo:** se elige técnica y profundidad según el riesgo del módulo.
- **Exploratorias estructuradas:** sesiones con objetivo y tiempo definidos, donde el tester aprende y prueba a la vez. Complementan la matriz formal; no la reemplazan.

Combinación recomendada (2.2.1): caja negra para interfaces, caja blanca para lógica y enfoque híbrido para integración. En un login: partición y valores límite para las credenciales; caminos básicos para la lógica de autenticación, cifrado y sesión. "Ninguna técnica por sí sola garantiza la calidad".

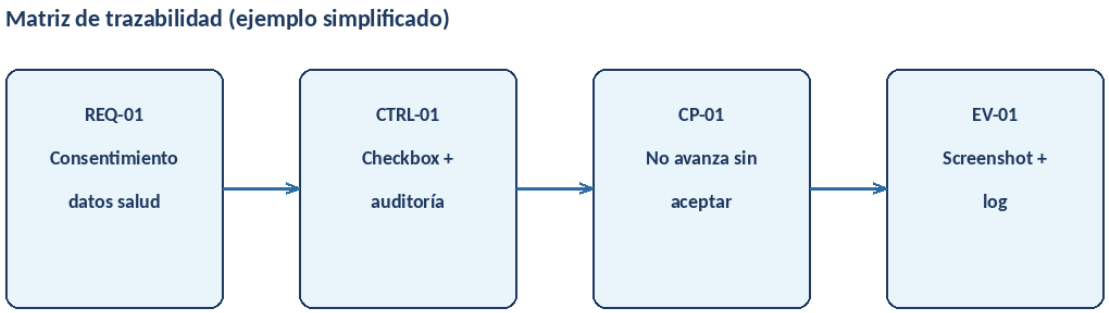
4.6 Proceso de creación de casos

1. **Identificar requisitos:** leer el ERS y extraer condiciones de prueba normales, excepcionales y de borde.

2. **Seleccionar la técnica:** según el contexto, la complejidad y los recursos.
3. **Diseñar casos detallados:** con todos los campos de 4.2.
4. **Revisar y refinar:** revisión por pares para eliminar ambigüedades, omisiones y redundancias.

Principios (2.2.1): calidad antes que cantidad; trazabilidad entre requisitos y casos; supuestos y limitaciones documentados; revisión por pares; herramientas adecuadas; medición y mejora continua.

2.6 Matriz de trazabilidad



Matriz de trazabilidad

4.9 Matriz de trazabilidad

Conecta cada requisito con sus casos y su resultado. Demuestra que ningún requisito quedó sin probar. En un informe de certificación serio se busca **cobertura completa** de los requisitos críticos del ERS (funcionales, no funcionales, reglas de negocio, seguridad y normativa).

Requisito	Descripción	Riesgo	Caso(s)	Tipo	Resultado	Incidente
RF-1.1	Inicio de sesión	Alto	CP-01, CP-SEG-001	Funcional / Seguridad	Aprobado / Falla	— / INC-xx
NFR-PERF-1	Tiempo de respuesta	Medio	CP-05	Rendimiento	…	…
RB-1	Regla de negocio	Alto	CP-02	Reglas de negocio	…	…

Reglas para que la matriz sirva:

- **Diseñado no es cubierto.** En la ejecución, un requisito está cubierto solo si su caso se ejecutó; si falló, queda cubierto con incidente.
- La cobertura se calcula como requisitos con al menos un caso ejecutado ÷ total de requisitos del ERS.
- Un caso puede cubrir varios requisitos, pero cada requisito debe tener al menos un caso.
- Los requisitos normativos (por ejemplo, registrar quién accede a un dato) también llevan caso.

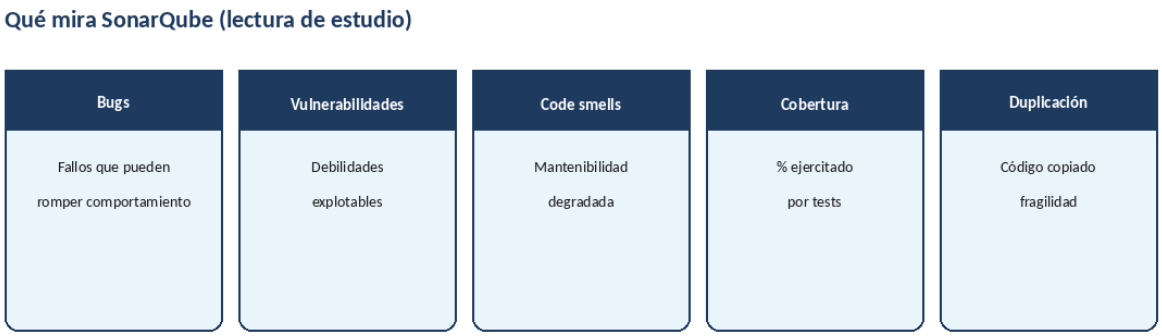
4.10 Herramientas de gestión de casos

TestRail, Jira + Zephyr, Azure Test Plans, qTest, TestLink y PractiTest. Beneficios: documentación centralizada con versiones, reutilización, reportes, trazabilidad entre requisitos, casos y defectos, integración con

automatización y colaboración. En el curso puede bastar una planilla bien estructurada, siempre que mantenga la trazabilidad.

> **Repaso 4.** Diseña los casos de valores límite para "edad entre 18 y 65". Calcula V(G) para una función con tres `if` independientes. Arma una tabla de decisión para "rol (admin/médico/paciente) × recurso (propio/ajeno)". ¿Qué diferencia hay entre un escenario y un caso?

2.7 Pruebas estáticas y SonarQube



Qué mira SonarQube

5.1 Qué es una prueba estática

Es el análisis del código fuente, del diseño o de la documentación **sin ejecutar el programa**. Es preventiva: encuentra defectos, vulnerabilidades y malas prácticas antes de que se transformen en fallas. Sus beneficios son la detección temprana, la reducción de costos de depuración y mantenimiento, la mejora de la legibilidad y la mantenibilidad, y el apoyo al cumplimiento normativo y a las auditorías.

	Estáticas	Dinámicas
¿Ejecutan el sistema?	No	Sí
Objeto	Código, diseño, documentación	Software en funcionamiento
Técnicas	Revisión, inspección, *walkthrough*, análisis automatizado (SAST)	Funcionales, DAST, carga, usabilidad
Resultado	Defectos con ubicación exacta	Fallas observadas y su comportamiento

Son complementarias: el análisis estático señala dónde puede haber un problema y la prueba dinámica confirma si se puede explotar.

5.2 Técnicas de revisión manual

Técnica	Cómo funciona	Formalidad
Revisión manual	Un revisor experto lee el código o la documentación	Baja

Técnica	Cómo funciona	Formalidad
Walkthrough	El autor guía al equipo por el código o el diseño	Media
Inspección	Revisión formal con roles (moderador, autor, lector, registrador), lista de verificación y registro de defectos	Alta
Análisis automatizado	Una herramienta busca patrones de error y vulnerabilidades	Automática

La revisión manual encuentra lo que las herramientas no ven: errores de **lógica de negocio** y de **autorización**. Ejemplos típicos: un endpoint que exige token pero no verifica el rol; un recurso que se entrega sin comprobar que pertenece al usuario; una eliminación física de datos que deberían conservarse; un registro de auditoría que no incluye las lecturas.

5.3 Obtención de evidencias en pruebas estáticas

1. **Documentar hallazgos:** cada descubrimiento de la revisión de código y diseño.
2. **Registrar defectos:** con detalle suficiente para corregirlos.
3. **Estandarizar formatos:** plantillas IEEE 829 para consistencia y trazabilidad.

Un registro de defecto estático contiene: descripción, **ubicación exacta (archivo y línea)**, regla o criterio incumplido, gravedad, recomendación y responsable. Se gestiona en Jira, Redmine, una hoja de cálculo compartida o el propio reporte exportado de la herramienta.

5.4 SonarQube: qué es y cómo se usa

SonarQube es una plataforma de análisis estático que detecta problemas de **seguridad** (vulnerabilidades), **confiabilidad** (bugs) y **mantenibilidad** (*code smells*) en más de 25 lenguajes, se integra en *pipelines* CI/CD y permite definir **Quality Gates** (umbrales de aprobación). Su documentación es bibliografía obligatoria del curso.

Flujo del laboratorio (2.3.1):

1. **Instalar y configurar** SonarQube Server (en el curso, como contenedor Docker).
2. **Importar el proyecto:** crear el proyecto y generar el token de análisis.
3. **Definir reglas:** perfil de calidad (*Quality Profile*) y Quality Gate.
4. **Escanear** el código del backend y del frontend con SonarScanner. Puede tardar de minutos a horas según el tamaño.
5. **Visualizar resultados** en el panel.
6. **Identificar defectos** y áreas críticas.
7. **Registrar evidencia manual:** exportar el reporte, complementarlo con anotaciones de contexto, comentar cada defecto relevante, documentar la acción correctiva y asignar un responsable.

5.5 Cómo leer el panel de SonarQube

Métrica	Qué mide	Cómo interpretarla
Bugs / Confiabilidad	Código que probablemente produce un comportamiento incorrecto	Calificación de A (sin problemas) a E (al menos un problema bloqueante)
Vulnerabilidades / Seguridad	Código explotable	Cualquier vulnerabilidad de severidad alta pasa a incidente
Security Hotspots	Código sensible que **requiere revisión humana**; no es necesariamente una vulnerabilidad	Se revisa y se marca como seguro o a corregir. El porcentaje revisado es una condición del Quality Gate
Code Smells / Mantenibilidad	Código difícil de mantener (duplicado, complejo, mal nombrado)	Afecta la mantenibilidad; rara vez es crítico por sí solo
Deuda técnica	Tiempo estimado para corregir los problemas de mantenibilidad	Se compara con el tamaño del proyecto
Cobertura	Porcentaje de código ejecutado por pruebas automatizadas	Si no hay pruebas unitarias aparece 0 %, y eso ya es un hallazgo
Duplicaciones	Porcentaje de líneas duplicadas	Aumenta el costo de corregir defectos
Complejidad	Ciclomática y cognitiva por función o archivo	Conecta con V(G) del punto 4.4

Severidad de los problemas. Según el modo configurado en la versión instalada, SonarQube muestra la clasificación clásica (*Bug*, *Vulnerability*, *Code Smell*, con severidades *Blocker*, *Critical*, *Major*, *Minor* e *Info*) o la clasificación por cualidad del software (Seguridad, Confiabilidad y Mantenibilidad, con severidades *Blocker*, *High*, *Medium*, *Low* e *Info*). En el informe conviene declarar cuál se usó.

Quality Gate "Sonar way" (el predeterminado en SonarQube Server 2025). Se aplica al **código nuevo** y falla si:

- se introducen problemas nuevos;
- no se revisó el 100 % de los *Security Hotspots* nuevos;
- la cobertura del código nuevo es menor a 80 %;
- la duplicación del código nuevo supera el 3 %.

Matiz importante: en un proyecto heredado analizado por primera vez casi todo es "código general" y no "código nuevo", así que el Quality Gate puede aparecer aprobado aunque haya muchos problemas. Para el informe se lee la pestaña de **código general* (Overall Code)* y se explica esa diferencia.

5.7 Beneficios, retos y recomendaciones (2.3.1)

Beneficios	Retos	Recomendaciones
Detección temprana	Disciplina y formación del equipo	Enfoque híbrido: herramienta más revisión manual
Mejora continua del código	Resistencia cultural a las revisiones	Capacitación continua
Apoyo al cumplimiento y a la auditoría	Coordinar herramientas automáticas con revisiones manuales	Procedimientos claros de registro, seguimiento y cierre
	Falsos positivos	Integrar el análisis en CI/CD para tener retroalimentación inmediata

El material muestra un ejemplo de resultado (15 bugs, 30 *code smells* y 5 vulnerabilidades en un proyecto Java) y recomienda registrar manualmente los defectos críticos para priorizarlos y darles seguimiento en las siguientes iteraciones.

> **Repaso 5.** ¿Qué diferencia hay entre una vulnerabilidad y un *Security Hotspot*? ¿Por qué un Quality Gate aprobado no significa que el código sea seguro? Toma un problema de SonarQube y llévalo hasta una norma.

2.8 Ejecución, bitácora e incidentes

6.1 Preparación del entorno (2.3.2, paso 1)

Antes de ejecutar se documenta:

Aspecto	Qué registrar
Hardware y software	Sistema operativo, navegador y versión, base de datos y versión, versión de Docker
Datos de prueba	Scripts de base de datos cargados; usuarios semilla; datos **sintéticos**
Herramientas de soporte	Capturador de pantalla, acceso a logs, herramientas de prueba listas
Versión del software	Commit o etiqueta de la versión probada

6.2 Ciclos de ejecución

Ciclo	Objetivo	Regla
Ciclo 1: smoke test	Verificar funciones básicas (login, carga de la página principal)	Si falla, la ejecución se detiene: el resto queda **Bloqueado**
Ciclo 2: ejecución de casos	Seguir el guion de cada caso, comparar obtenido con esperado y registrar evidencia de éxitos y fallas	Se respeta el orden de prioridad por riesgo
Ciclo 3, si corresponde: re-test y regresión	Confirmar correcciones y que no se rompió otra cosa	Solo después de una corrección

6.3 Estados de un caso

Estado	Significado
Aprobado (Pass)	El resultado obtenido coincide con el esperado y cumple el criterio de aceptación
Falla (Fail)	El resultado no coincide; se abre un reporte de defecto
Bloqueado	No se pudo ejecutar porque otro defecto o el entorno lo impiden
No ejecutado	Se diseñó pero no se ejecutó; debe justificarse en el informe

6.4 Reporte de defecto (bug report)

Campo	Ejemplo
ID	BUG-001 / INC-01
Título	Contraseñas almacenadas sin hash
Caso y requisito relacionados	CP-SEG-002 · requisito de almacenamiento seguro de credenciales
Severidad	Crítica
Prioridad	Alta
Entorno	Versión, navegador, contenedor
Pasos para reproducir	Instrucciones exactas para que el desarrollador vea el problema
Resultado esperado / obtenido	Hash bcrypt / texto plano
Evidencia	Captura, log, consulta, archivo y línea
Estado	Abierto / En corrección / Corregido / Verificado / Cerrado

Severidad y prioridad no son lo mismo. La severidad mide el **impacto técnico** del defecto; la prioridad mide la **urgencia** de corregirlo según el negocio. Un error ortográfico en el nombre de la empresa en la página principal tiene severidad baja y prioridad alta; una falla en una función que casi nadie usa puede tener severidad alta y prioridad media.

6.5 Escalas de severidad que usa la unidad

El material usa **tres escalas distintas**. En el informe de certificación conviene declarar la escala usada y explicar equivalencias si hay varias fuentes.

Escala del informe	Lab. ejecución	Lab. certificación	Descripción	Tratamiento
Crítico	Crítica	Críticos	Impide el funcionamiento del sistema o de una función esencial; bloquea el flujo principal. Impacto muy alto	Remediación inmediata
Grave	Mayor	Mayores	Afecta funcionalidades importantes, pero el sistema funciona parcialmente; existe una alternativa	Corregir con urgencia
Moderado	Menor	Menores	Afecta funciones secundarias o casos específicos sin detener el flujo principal	Próxima iteración
Leve	Cosmética	(Menores)	Apariencia, estética o texto; no afecta la funcionalidad	Según disponibilidad

Criterio para fallas de seguridad. La actividad 2.4.2 ubica las fallas de seguridad (por ejemplo, una inyección SQL) y las caídas del sistema como **críticas**. Para clasificar una falla de seguridad conviene preguntarse:

1. ¿Expone datos personales o sensibles?
2. ¿Permite acceder o actuar sin autorización?
3. ¿Se puede explotar sin credenciales o con poco esfuerzo?
4. ¿Incumple un requisito legal directo?

Si la respuesta a la 1 o a la 2 es sí y la 3 también, la falla es crítica aunque el sistema "siga funcionando".

6.6 Bitácora de ejecución

ID caso	Nombre	Estado	Defecto	Observaciones	Fecha / responsable
CP-001	Login con SQL	Aprobado	N/A	Mensaje de error genérico	dd-mm-aaaa / nombre
CP-002	Registro de usuario	Falla	BUG-002	El botón "Enviar" no responde en Chrome	…
CP-003	Compra de producto	Bloqueado	N/A	No se pudo probar por la falla de CP-002	…

El formato de informe agrega dos columnas recomendadas en el detalle de ejecución: **evidencia** y **fecha y persona que ejecuta**.

6.7 Informe resumen de pruebas

Responde con números: casos diseñados, ejecutados, aprobados, fallidos, bloqueados y no ejecutados; defectos por severidad; cobertura de requisitos. Cierra con una recomendación: "¿Está listo para producción?" o "¿Requiere un nuevo ciclo de corrección?".

Fórmulas útiles:

- Porcentaje de ejecución = ejecutados ÷ diseñados × 100.
- Porcentaje de aprobación = aprobados ÷ ejecutados × 100.
- Cobertura de requisitos = requisitos con caso ejecutado ÷ requisitos del ERS × 100.
- Densidad de defectos por módulo = defectos ÷ módulo (ayuda a ubicar dónde se agrupan).

> **Repaso 6.** ¿Cuándo se marca un caso como Bloqueado y no como Falla? Da un ejemplo de defecto con severidad baja y prioridad alta. ¿Qué escala de severidad usarías en el informe de certificación y por qué?

2.9 Informe de certificación y ética

7.1 Para qué sirve el informe

La presentación 2.4.1 lo compara con un "título de propiedad" de la confianza: es el documento oficial que valida el cumplimiento de requisitos, sirve de evidencia ante reguladores, clientes y partes interesadas, y

puede revisarse en auditorías posteriores. Un informe impreciso o incompleto afecta la reputación de quien lo emite y su validez.

7.2 Estándares de referencia

Norma	Ámbito
ISO/IEC 17021-1	Requisitos para organismos que auditan y certifican sistemas de gestión
ISO/IEC 17065	Requisitos para organismos que certifican productos, procesos y servicios
ISO 19011	Directrices para auditar sistemas de gestión (la edición 2018 aplica a cualquier sistema de gestión, no solo a calidad y ambiente)
IAF MD 4	Documento obligatorio del Foro Internacional de Acreditación sobre el uso de tecnologías de información y comunicación (TIC) en auditorías y evaluaciones

En el curso estas normas sirven como modelo de **cómo se estructura y se valida un informe de certificación**. Un informe de pruebas de un estudiante no "certifica ISO": usa esa lógica para emitir un dictamen técnico.

7.3 Estructura del informe de certificación (2.4.1)

#	Sección	Contenido
1	**Identificación y portada**	Responsable, cliente, alcance, norma o criterio aplicado, fechas, código único del informe
2	**Resumen ejecutivo**	1 a 2 páginas: tipo de evaluación, fechas, equipo, hallazgos positivos, no conformidades críticas y decisión con justificación técnica, en lenguaje claro
3	**Metodología**	Planificación → ejecución → análisis → reporte. Enfoque basado en riesgos, muestreo y criterios de evaluación
4	**Hallazgos**	Conformidades, observaciones, no conformidades menores y mayores
5	**Conclusiones y recomendación**	Evaluación global, decisión (otorgar, denegar u otorgar con condiciones), condiciones y plazos, seguimiento
6	**Anexos y evidencias**	Plan, registros de evidencia, matriz de trazabilidad, competencias del equipo

Clasificación de hallazgos

Hallazgo	Significado
Conformidad	Evidencia de cumplimiento; muestra fortalezas
Observación	Oportunidad de mejora sin incumplimiento
No conformidad menor	Desviación que no compromete la efectividad global
No conformidad mayor	Incumplimiento que afecta la capacidad del sistema de lograr sus resultados

Cómo se documenta una no conformidad: (1) descripción clara del requisito incumplido y de la desviación; (2) evidencia objetiva con referencias, fechas y ubicaciones; (3) referencia normativa con el número de cláusula o artículo; (4) clasificación justificada como mayor o menor. Omitir la cláusula "para que el cliente lo entienda mejor" es un error, no una simplificación (pausa activa de 2.4.1).

7.4 Requisitos de la evidencia y de la redacción

Evidencia	Significado
Verificable	Un tercero puede comprobarla de forma independiente
Trazable	Está numerada, fechada y referenciada en el texto
Suficiente	Su cantidad y calidad son proporcionales al riesgo y al alcance

Lenguaje: objetivo, preciso, claro, neutral y consistente. Se evita la primera persona, las opiniones, las conclusiones sin evidencia, las descripciones vagas de no conformidades, la falta de referencias normativas y las inconsistencias entre secciones.

Errores que invalidan un informe: hallazgos sin evidencia objetiva; referencias normativas incorrectas o desactualizadas; falta de firmas o aprobaciones; contradicciones entre el resumen ejecutivo y las conclusiones.

7.5 Control documental

- **Codificación:** clave del titular, área, número correlativo, versión y clasificación (ejemplo del material: `BOCE-2024-0125-V1-INICIAL`).
- **Registro de cambios:** fecha, descripción, tipo (de forma o de fondo), responsable y aprobación.
- **Revisión y aprobación:** elaboración por el líder → revisión del equipo → validación técnica independiente → aprobación final.
- **Confidencialidad:** clasificar la información, acuerdos de confidencialidad, almacenamiento seguro, distribución controlada, retención y destrucción. Un informe de seguridad describe vulnerabilidades explotables, así que su circulación debe limitarse.
- **Mejora continua del reporte:** fijar objetivos, monitorear, identificar mejoras, implementarlas y verificar resultados.

8.1 Cómo se usa la normativa en esta unidad

La unidad no introduce leyes nuevas desde cero: reutiliza el marco de la EA1 (carpeta de normativas del curso) y exige **aplicarlo a los resultados de las pruebas**. El análisis ético y normativo del informe y las recomendaciones de la presentación se construyen con esta secuencia:

```
Hallazgo técnico → Riesgo para las personas → Norma y artículo → Quién está obligado → Recomendación
```

Tres reglas:

1. **Citar el artículo, no solo la ley.** "Ley 19.628" no basta; "Ley 19.628, art. 11 (deber de diligencia en el cuidado de los datos)" sí.

2. **Separar lo vigente de lo que viene.** No atribuir a una ley derechos o sanciones que están en otra, ni presentar como vigente una norma que todavía no rige.
3. **Identificar al obligado.** Muchas leyes no obligan al software, sino a la organización que lo opera. El software debe **permitir** que esa organización cumpla.

8.2 Normas del curso (aplican a cualquier caso)

Norma	Estado (al 19-10-2026)	Qué aporta al análisis de pruebas
Ley 19.628, protección de la vida privada	Vigente en su texto actual	Define datos personales y sensibles (art. 2); deber de secreto (art. 7); tratamiento de datos sensibles (art. 10); deber de cuidar los datos con la debida diligencia (art. 11); indemnización por tratamiento indebido (art. 23)
Ley 21.719, protección de datos personales	Publicada el 13-12-2024; **entra en vigencia el 01-12-2026**	Reemplaza gran parte del texto de la Ley 19.628. Agrega deberes de seguridad (art. 14 quinquies), reporte de vulneraciones (art. 14 sexies), infracciones leves, graves y gravísimas, y multas de hasta 5.000, 10.000 y 20.000 UTM (art. 35 del texto nuevo). Crea la Agencia de Protección de Datos Personales. Incorpora principios como licitud, finalidad, proporcionalidad, calidad, responsabilidad, seguridad, transparencia y confidencialidad
Ley 21.663, Ley Marco de Ciberseguridad	Vigente; la Agencia Nacional de Ciberseguridad (ANCI) opera desde 2025	Obliga a las instituciones que prestan **servicios esenciales** y a los **operadores de importancia vital**. Exige reportar incidentes al CSIRT Nacional (alerta temprana en 3 horas). Multas según gravedad, mayores para operadores de importancia vital (art. 40)
Ley 21.459, delitos informáticos	Vigente	Tipifica el ataque a la integridad de un sistema, el acceso ilícito, la interceptación ilícita y el ataque a la integridad de los datos. Sirve para explicar el **impacto** de una vulnerabilidad y para fijar los **límites éticos** de las pruebas
Ley 17.336, propiedad intelectual	Vigente	Uso de software y componentes de terceros; licencias de dependencias; sanciones (arts. 78 y 79)
Ley 19.799 y Decreto 181, documento y firma electrónica	Vigente	Aplica cuando el sistema emite o firma documentos electrónicos con valor legal
Ley 21.180, transformación digital del Estado	Vigente	Aplica a sistemas que se relacionan con organismos públicos

Norma	Estado (al 19-10-2026)	Qué aporta al análisis de pruebas
Política Nacional de Ciberseguridad	Política pública	Marco general; orienta, pero no impone obligaciones directas a un sistema privado
Manual de Accesibilidad Web (Gobierno de Chile)	Guía técnica	Base para requisitos y pruebas de accesibilidad y usabilidad

8.3 Normas sectoriales: dependen del dominio del caso

El docente puede pedir normas propias del negocio. La pregunta guía es: **¿qué ley regula la actividad que el software apoya?**

Dominio	Normas sectoriales que suelen aparecer	Ejemplo de prueba que generan
Salud (por ejemplo, Cuidarte+: pacientes, exámenes, documentos clínicos)	Ley 20.584 de derechos y deberes de los pacientes (ficha clínica como dato sensible, acceso restringido y conservación, arts. 12 y 13); Ley 21.668 (interoperabilidad de fichas clínicas); Decreto 41 de 2012 del MINSAL, reglamento de fichas clínicas (registro de quién accede y cuándo, respaldos); normas de atención de urgencia del DFL 1 de 2005 del MINSAL (arts. 141 y 173)	Cada lectura o descarga de un examen queda registrada con usuario y fecha; un profesional no vinculado a la atención no accede a la ficha; los datos no se eliminan físicamente antes del plazo de conservación
Gestión empresarial / pymes (por ejemplo, Atlas: clientes, contratos y documentos)	Ley 19.628 y Ley 21.719 para los datos de clientes; Ley 19.799 si los contratos se firman electrónicamente	Un editor no modifica contratos; los datos de un cliente no se ven desde otra empresa
Banca y finanzas (caso de la actividad 2.1.2)	Normativa del regulador financiero; Ley 21.663 si es operador de importancia vital	Bloqueo tras intentos fallidos; invalidación de tokens; cifrado en tránsito

En un caso de salud, la normativa sanitaria es la que regula de forma específica el dato clínico, así que va primero en el análisis. Omitirla es una falta de cobertura normativa.

8.4 Principios éticos aplicados a las pruebas

Principio	Qué significa en la práctica
Autorización y alcance	Las herramientas de seguridad (ZAP, Burp, SQLMap, Nikto, John the Ripper) se usan **solo** contra el entorno propio y autorizado (el Docker local del caso). Usarlas contra sistemas ajenos es la conducta que sanciona la Ley 21.459
Datos sintéticos	Nunca usar datos reales de personas (RUT, diagnósticos, contratos), aunque el entorno sea local
Honestidad en el reporte	Reportar todos los hallazgos, también los incómodos. Ocultar un defecto para mejorar el dictamen es una falta profesional
Proporcionalidad	Demostrar la vulnerabilidad sin causar más daño del necesario (no borrar datos ni saturar un entorno compartido)

Principio	Qué significa en la práctica
Confidencialidad	El informe describe vulnerabilidades: su distribución se controla
Foco en las personas	El impacto se mide en las personas afectadas (pacientes, adultos mayores, clientes), no solo en el sistema
Independencia	El juicio de QA no se subordina a plazos ni a la presión por aprobar
Mejora continua	Cada hallazgo termina en una recomendación concreta y verificable

8.5 Ejemplo de análisis ético-normativo bien construido

Elemento	Contenido
Hallazgo	Las contraseñas se guardan y comparan en texto plano (archivo y línea)
Riesgo	Si se filtra la base de datos, todas las cuentas quedan expuestas; muchas personas reutilizan contraseñas
Norma vigente	Ley 19.628, art. 11: el responsable debe cuidar los datos con la debida diligencia
Norma próxima	Art. 14 quinquies del texto que introduce la Ley 21.719 (medidas de seguridad), desde el 01-12-2026
Requisito del ERS	Contraseñas con hash bcrypt
Obligado	La organización que opera el sistema; el desarrollador entrega un sistema que no le permite cumplir
Severidad	Crítica
Recomendación	Aplicar bcrypt, forzar el cambio de las contraseñas existentes y agregar un caso de regresión
Impacto en el dictamen	Con este defecto abierto el sistema no puede certificarse

Repaso 8. ¿Por qué no se debe citar la Ley 21.719 como vigente en octubre de 2026? ¿A quién obliga la Ley 21.663? Nombra dos principios éticos que limitan el uso de SQLMap.

2.10 Catálogo de herramientas

9.1 Herramientas del laboratorio de la Evaluación 2

Lista de herramientas disponibles en Docker para los casos A y B, según el material de Eva_02.

Herramienta	Categoría	Qué hace	Tipo de prueba / requisito que respalda	Evidencia que produce
SonarQube	SAST / calidad	Analiza el código sin ejecutarlo; detecta bugs, vulnerabilidades, *hotspots* y *code smells*	Análisis estático (IL2.3); mantenibilidad; seguridad del código	Panel, reporte exportado, archivo y línea

Herramienta	Categoría	Qué hace	Tipo de prueba / requisito que respalda	Evidencia que produce
OWASP ZAP	DAST	Proxy y escáner de aplicaciones web; rastrea la app y lanza pruebas pasivas y activas	Seguridad de la app desplegada (configuración, cabeceras, inyección, XSS). Para llegar a las funciones internas se configura con sesión iniciada	Reporte HTML o JSON con alertas y nivel de riesgo
Burp Suite	Proxy / pruebas manuales	Intercepta y modifica peticiones HTTP y permite repetirlas con cambios	Pruebas manuales de autorización (cambiar un ID, quitar el token, cambiar el rol), sesiones y lógica de negocio	Petición y respuesta capturadas
Nikto	Escáner de servidor web	Busca configuraciones inseguras, archivos expuestos y versiones conocidas del servidor	Seguridad de configuración del servidor	Listado de hallazgos por URL
SQLMap	Inyección SQL	Detecta de forma automatizada inyecciones SQL en parámetros	Validación de entradas en el login y en endpoints con parámetros	Resultado por parámetro (vulnerable o no)
John the Ripper	Auditoría de contraseñas	Intenta recuperar contraseñas a partir de sus hashes con diccionarios y fuerza bruta	Fortaleza de la política de contraseñas y del algoritmo de hash	Contraseñas recuperadas y tiempo empleado
JMeter	Rendimiento	Simula usuarios concurrentes y mide tiempos, rendimiento y errores	Carga y estrés (requisitos de rendimiento y disponibilidad)	Reporte HTML con percentiles, rendimiento y tasa de error
Selenium	Automatización funcional	Controla el navegador para repetir flujos de usuario	Funcionales, regresión y compatibilidad entre navegadores	Script, registro de ejecución y capturas

Lectura correcta de los resultados:

- **ZAP y Nikto** generan alertas que pueden ser falsos positivos; cada alerta relevante se confirma a mano antes de convertirla en incidente.
- **SQLMap** se ejecuta solo contra el entorno propio. Si concluye que un parámetro no es vulnerable, ese resultado es evidencia de un caso aprobado.
- **John the Ripper** solo tiene sentido si hay hashes. Si las contraseñas están en texto plano, el hallazgo es todavía más grave y la herramienta no hace falta para demostrarlo.
- **JMeter**: se reportan percentiles (por ejemplo, el percentil 90) y no solo el promedio. Si el código tiene retardos artificiales, se mide con y sin ellos y se explica la diferencia.

9.2 Otras herramientas mencionadas en la unidad

Categoría	Herramientas	Uso
Automatización funcional	Cypress, Playwright, Appium (móvil)	Alternativas a Selenium
Gestión de casos	TestRail, Jira + Zephyr, Azure Test Plans, qTest, TestLink, PractiTest	Repositorio de casos, ejecución y trazabilidad
Seguimiento de defectos	Jira, Bugzilla, Redmine	Ciclo de vida de los defectos
Infraestructura	Docker, Docker Compose	Entorno reproducible
Pruebas de API	Swagger UI (`/docs`), Postman	Pruebas funcionales y de autorización sobre la API

9.3 Cómo justificar una herramienta

La evaluación premia la justificación. Una fórmula simple:

> "Se usa **[herramienta]** para **[tipo de prueba]** sobre **[requisito o riesgo]**, porque **[capacidad concreta]**. Su resultado se registra como **[evidencia]** en el caso **[ID]**."

Ejemplo: "Se usa JMeter para la prueba de carga del requisito de tiempo de respuesta, porque permite simular 200 usuarios concurrentes y medir el percentil 90. El reporte HTML se adjunta como evidencia del caso CP-05".

> **Repaso 9.** ¿Qué herramienta usarías para probar que un usuario no puede ver un registro de otro usuario? ¿Por qué ZAP sin sesión iniciada da una cobertura limitada? ¿En qué se diferencian ZAP y Nikto?

2.11 Chuleta de la Unidad 2

Tema	Recuerda
Plan de pruebas	Alcance, riesgos, niveles/tipos, entrada/salida, evidencias
Riesgo	Probabilidad × impacto → prioriza qué probar primero
Docker	Imagen fija + reproducible; no es «magia de seguridad» por sí sola
Caso de prueba	ID, precondiciones, pasos, datos, esperado, trazabilidad
Caja negra	Desde especificación (equivalencia, límites, decisión)
Caja blanca	Desde estructura (cobertura de código/caminos)
Trazabilidad	REQ ↔ caso ↔ evidencia
Estático	Sin ejecutar: revisiones + SAST (SonarQube)
Bitácora	Qué se ejecutó, resultado, quién, cuándo
Certificación	Informe con evidencias, alcance, limitaciones, ética

2.12 Preguntas de autoevaluación

1. **¿Qué es un plan de pruebas?** Documento que define alcance, objetivos, enfoque, recursos, calendario, riesgos y criterios de entrada/salida para las pruebas.

2. **¿Para qué sirven los criterios de suspensión?** Para saber cuándo detener la ejecución (p. ej. ambiente inestable) y no quemar esfuerzo en ruido.
3. **¿Qué es una estrategia basada en riesgos?** Priorizar pruebas según probabilidad e impacto de fallo, no solo «probar todo un poco».
4. **¿Por qué Docker ayuda al laboratorio de pruebas?** Fija runtime y dependencias; reduce el «en mi máquina funciona».
5. **¿Qué campos mínimos tiene un caso de prueba?** Identificador, título, precondiciones, pasos, datos, resultado esperado, y enlace a requisito.
6. **¿Partición de equivalencia qué busca?** Agrupar entradas que deberían tratarse igual y probar representantes, no todos los valores.
7. **¿Análisis de valores límite dónde mira?** En los bordes de las particiones (justo dentro/fuera), donde más fallan los defectos.
8. **¿Cobertura de decisiones qué exige?** Ejercitar verdaderas y falsas de cada decisión relevante del código.
9. **¿Para qué una matriz de trazabilidad?** Demostrar que cada requisito crítico tiene caso(s) y evidencia.
10. **¿Prueba estática ejecuta el programa?** No. Revisa artefactos (código, docs) sin correrlo.
11. **¿Qué miran bugs, vulnerabilities y smells en SonarQube?** Fallos de comportamiento, debilidades de seguridad, y problemas de mantenibilidad.
12. **¿Qué va en un reporte de defecto?** Pasos, esperado vs obtenido, severidad, entorno, adjuntos; sin eso no es accionable.
13. **¿Severidad frente a prioridad?** Severidad = impacto técnico/negocio del fallo; prioridad = urgencia de arreglo.
14. **¿Qué no puede faltar en un informe de certificación?** Alcance, resultados, evidencias, limitaciones, conclusión y responsables.
15. **¿Qué principio ético aplica al manejar datos de prueba reales?** Minimizar datos personales, anonimizar cuando baste, y no exponer secretos en evidencias.

2.13 Ejercicios resueltos

2.13.1 Ejercicio 1: límites

Campo edad acepta enteros 18–65 inclusive. Diseña particiones y valores límite de estudio.

Solución. Inválido bajo: ≤17 (límite 17). Válido: 18–65 (límites 18 y 65). Inválido sobre: ≥66 (límite 66). Casos mínimos: 17, 18, 65, 66 (y opcional un medio válido, p. ej. 40).

2.13.2 Ejercicio 2: leer SonarQube

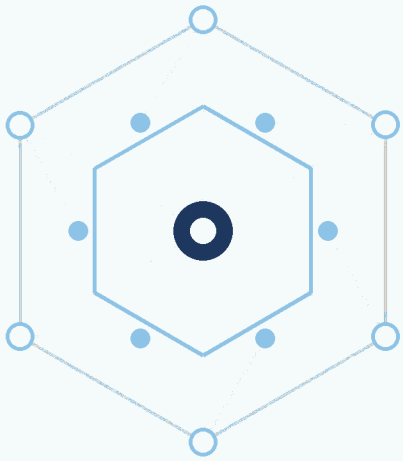
El panel muestra 0 bugs, 3 vulnerabilities *major*, 120 smells y 15 % coverage. El quality gate exige 0 vulnerabilidades y coverage ≥ 60 %. ¿Pasa?

Solución. No. Hay vulnerabilidades > 0 y coverage 15 % < 60 %. Aunque bugs = 0, el gate falla por seguridad y por cobertura.

2.13.3 Ejercicio 3: trazabilidad rota

REQ-07 «solo rol admin importa usuarios» tiene caso CP-07, pero la evidencia adjunta es un login de usuario *viewer* exitoso sin intentar la acción restringida. ¿Qué falta?

Solución. La evidencia no demuestra el requisito: falta el intento de importar con *viewer* (debe denegar) y con *admin* (debe permitir), más registro/autorización. Hay que reejecutar y adjuntar evidencia alineada al esperado del caso.



EVALUACIÓNEA3

Unidad 3. Automatización, DevSecOps

y aseguramiento continuo

Cómo escalar el enfoque: pentesting ético, controles SAST/DAST en un pipeline CI/CD, automatización funcional, pruebas de carga y estrés, y el mantenimiento del aseguramiento continuo bajo cambio constante.

Viene de la Unidad 2: cuando los casos y la certificación ya existen, la Unidad 3 los integra al flujo de entrega para que calidad y seguridad no dependan de una corrida manual puntual.

EA3

UNIDAD 3

Contenidos de la unidad

3.1 Pentesting: pruebas de penetración éticas
3.2 Pipeline CI/CD con controles de seguridad
3.3 Automatización de pruebas funcionales y mejora de scripts
3.4 Pruebas de carga, estrés y pico
3.5 Mantenimiento del aseguramiento continuo
3.6 Chuleta de la Unidad 3
3.7 Preguntas de autoevaluación
3.8 Ejercicios resueltos

Unidad 3. Automatización, DevSecOps y aseguramiento continuo

La Unidad 3 escala lo aprendido: las pruebas y los controles de seguridad dejan de ser un evento puntual y pasan a integrarse en el flujo de entrega. Aquí aparecen el **pentesting** ético, el **pipeline CI/CD** con **SAST** y **DAST**, la automatización funcional, las pruebas de **carga**, **estrés** y **pico**, y el mantenimiento del aseguramiento continuo.

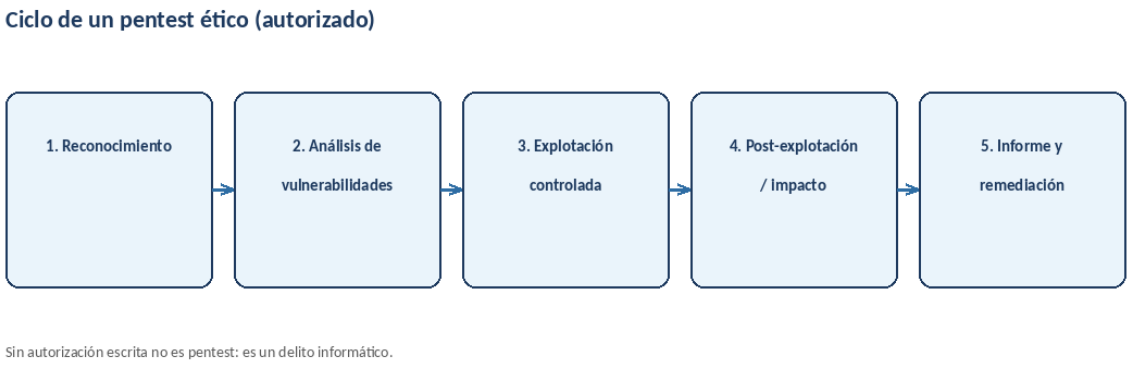
Cada término se define al aparecer; cada bloque cierra con ideas clave. Al final: chuleta, quince preguntas con respuesta y tres ejercicios resueltos.

3.1 Pentesting: pruebas de penetración éticas

3.1.1 Qué es y qué no es

El **pentesting** (prueba de penetración) es un ataque informático **simulado y controlado** realizado por profesionales autorizados. Su objetivo es identificar vulnerabilidades en sistemas, redes, aplicaciones y otros activos **antes** de que un actor malicioso las explote. A diferencia de un ataque real, se ejecuta con **consentimiento explícito**, metodologías estructuradas y fin de fortalecer la postura de seguridad.

Sin autorización escrita no es pentest: en Chile puede configurar un **delito informático** (Ley 21.459). El alcance, las ventanas de tiempo, los sistemas incluidos y excluidos, y las reglas de engagement deben quedar documentados.



Ciclo de un pentest ético

3.1.2 Objetivos estratégicos

Objetivo	En la práctica
Identificar vulnerabilidades	Descubrir fallos en sistemas, apps y configuraciones
Evaluar defensas	Probar controles y capacidad de respuesta
Proveer evidencias	Hallazgos reproducibles + recomendaciones de remediación
Apoyar cumplimiento	Insumo para auditorías (p. ej. enfoques tipo ISO 27001)

3.1.3 Tipos según conocimiento previo

Tipo	Qué sabe el equipo	Qué simula
Black box	Nada o casi nada del interior	Atacante externo realista
White box	Código, arquitectura, credenciales, docs	Evaluación profunda con máxima información
Grey box	Información parcial (p. ej. usuario básico)	Insider o atacante con acceso limitado

3.1.4 Metodología (fases)

1. **Reconocimiento:** mapear superficie de ataque (dominios, puertos, tecnologías).
2. **Análisis de vulnerabilidades:** identificar debilidades conocidas y sospechosas.
3. **Explotación controlada:** demostrar impacto sin destruir el negocio.
4. **Post-explotación:** ver hasta dónde se podría pivotar (con límites acordados).
5. **Informe y remediación:** hallazgo, evidencia, severidad, recomendación, retest.

> **Ojo:** un escaneo automático de vulnerabilidades **no** sustituye un pentest. El escáner lista CVEs posibles; el pentest demuestra explotabilidad en el contexto real, con criterio humano.

> **Nota por confirmar:** cifras de costo promedio de brechas varían por informe anual (IBM, Verizon, etc.); cítalas con año y fuente si las usas en un informe.

3.1.5 Herramientas y referencias habituales

En el material del curso aparecen, entre otras, referencias a metodologías tipo OWASP y a herramientas de apoyo (escáneres, proxies interceptores, frameworks de explotación en laboratorio). Lo importante no es memorizar logos: es saber **qué pregunta responde cada herramienta** y que el valor está en el **informe accionable**.

Además, el material de la unidad enfatiza: ¿Por qué el Pentesting es tu mejor inversión? El pentesting no es solo un "ataque simulado"; es la herramienta proactiva que te permite encontrar tus debilidades antes de que un actor malicioso las explote. Introducción al Pentesting: Definición y objetivos estratégicos ¿Qué es el Pentesting? El pentesting, o prueba de penetración, es un ataque informático simulado y controlado realizado por profesionales de seguridad autorizados. Su objetivo principal es identificar vulnerabilidades en sistemas, redes, aplicaciones web y otros activos digitales antes de que actores maliciosos puedan explotarlas. A diferencia de un ataque real, el pentesting se ejecuta con el consentimiento explícito de la organización propietaria del sistema, siguiendo metodologías estructuradas y con el fin último de fortalecer la postura de seguridad de la empresa. toda la información por sí mismo, Permite una evaluación exhaustiva y Simula amenazas internas o ataques desde la superficie de ataque hasta profunda de todos los componentes de empleados con acceso limitado las vulnerabilidades internas. Fase 2: Escaneo y Análisis Durante esta etapa, se emplean herramientas especializadas para identificar servicios activos, versiones de software y posibles vulnerabilidades presentes en la superficie de ataque.

Ideas clave del bloque 3.1

- Pentest = ataque autorizado, con alcance y reglas.
- Black / white / grey box cambian la información previa, no la ética.
- Informe sin evidencia reproducible no sirve para remediación.

3.2 Pipeline CI/CD con controles de seguridad

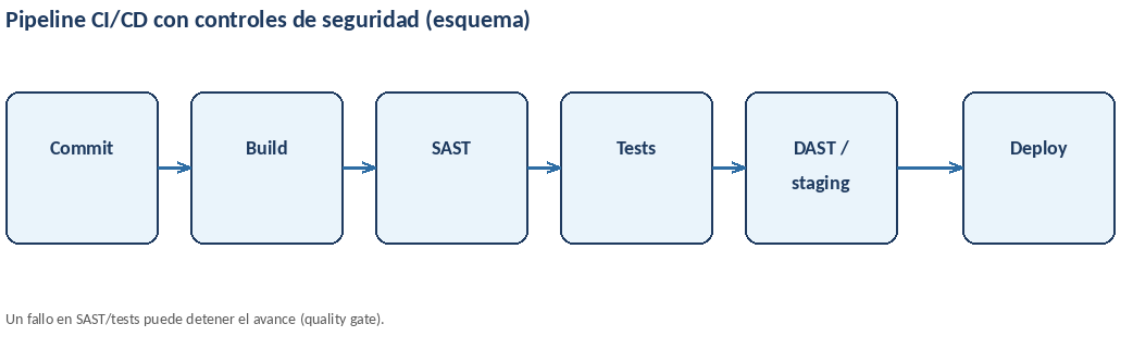
3.2.1 Qué es un pipeline CI/CD

CI (*Continuous Integration*) es la práctica de integrar cambios de código con frecuencia en un repositorio compartido, disparando compilaciones y pruebas automáticas. **CD** puede significar *Continuous Delivery* (siempre listo para desplegar) o *Continuous Deployment* (despliegue automático a producción si el pipeline está verde). Un **pipeline** es la secuencia automatizada de etapas (build → pruebas → análisis → despliegue).

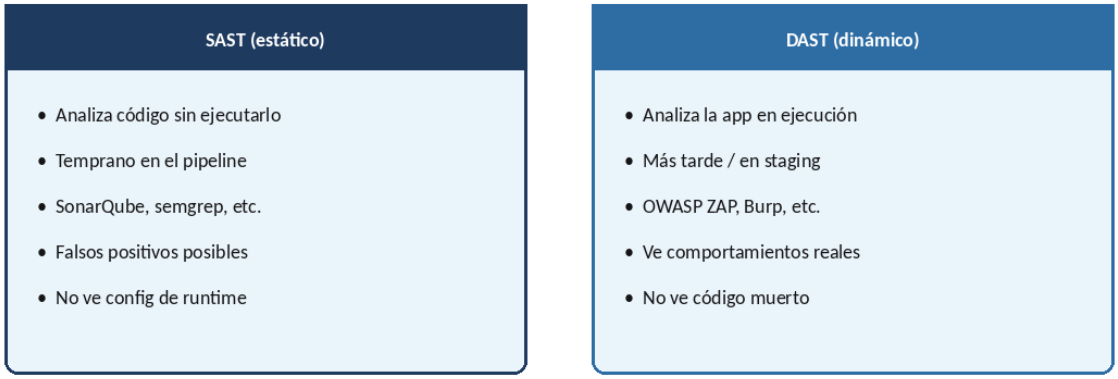
3.2.2 Seguridad en el pipeline (DevSecOps)

DevSecOps integra seguridad en el mismo flujo de desarrollo y operaciones, en lugar de auditar solo al final. Controles típicos:

Control	Qué hace	Cuándo suele correr
SAST	Analiza código/binarios **sin** ejecutar la app	Temprano (build)
DAST	Analiza la app **en ejecución**	Staging / ambiente desplegado
Dependencias	Busca CVEs en librerías	Build
Secrets scan	Detecta claves/tokens en el repo	Pre-commit / build
Quality gate	Bloquea el avance si no se cumplen umbrales	Tras análisis/tests



Pipeline CI/CD con controles



SAST frente a DAST

3.2.3 SAST frente a DAST

	SAST	DAST
Entrada	Código fuente / bytecode	App corriendo (HTTP, etc.)
Ventaja	Temprano, señala archivo/línea	Ve config real, auth, runtime
Límite	Falsos positivos; no ve runtime	Más tarde; no ve código muerto
Ejemplos de estudio	SonarQube, semgrep	OWASP ZAP, Burp

Ojo: «tenemos SonarQube» no equivale a «somos seguros». Hay que definir **quality gates** (qué fallos detienen el pipeline) y responsables de remediar.

Nota por confirmar: los umbrales exactos de coverage o severity los define el equipo/proyecto; el curso enseña el mecanismo, no un número mágico universal.

Además, el material de la unidad enfatiza: Fundamentos y Desafíos: Del testing manual a la necesidad de automatización. Fortalecimiento de la Estrategia de Calidad Para que un pipeline sea efectivo, debe seguir la Pirámide de Testing: • Base amplia de pruebas unitarias: Deben ser rápidas y económicas para proporcionar feedback inmediato. • Capa intermedia de integración: Validar la interacción entre componentes. • Vértice de pruebas E2E (End-to-End): Realizar validaciones exhaustivas del sistema completo, aunque sean menos numerosas. • Filosofía Fail Fast: Diseña el pipeline para que se detenga ante el primer fallo, evitando el desperdicio de recursos en etapas posteriores. Diseño de un Pipeline CI/CD con Controles de Seguridad Para dar inicio, debemos comprender que la automatización no es solo una cuestión de rapidez. En los entornos de desarrollo ágiles actuales, integrar controles de seguridad y automatizar pruebas es el estándar indispensable para mantener la calidad sin sacrificar la velocidad de entrega. No se trata solo de "hacer código", sino de asegurar que cada pieza de software sea robusta y segura desde su concepción.

Ideas clave del bloque 3.2

- CI integra y prueba seguido; el pipeline concreta esas etapas.
- SAST ≠ DAST: estático temprano frente a dinámico en runtime.

- Un control sin quality gate es decorativo.

3.3 Automatización de pruebas funcionales y mejora de scripts

3.3.1 Por qué automatizar (y qué no automatizar primero)

La **automatización de pruebas funcionales** usa scripts o herramientas (p. ej. **Selenium**) para ejercer flujos de usuario de forma repetible. Automatizar conviene cuando el flujo es estable, se repite en cada build y el costo de mantenimiento del script es menor que el de reejecutar a mano. No conviene automatizar primero lo que cambia cada semana o lo que aún no se entiende en manual.

3.3.2 Buenas prácticas de scripts

Práctica	Por qué importa
Selectores estables	Evitan que un cambio de CSS rompa todo
Datos de prueba controlados	Resultados reproducibles
Esperas explícitas	Menos *flaky tests* por carreras
Independencia entre casos	Un fallo no contamina el siguiente
Reportes legibles	La falla debe decir *qué* rompió

3.3.3 Mejora de código de prueba

Revisar scripts es parte del aseguramiento: eliminar duplicación, parametrizar datos, centralizar page objects o helpers, y alinear nombres con la matriz de trazabilidad. Un test ilegible es deuda: cuando falle, nadie sabrá si falló el producto o el propio test.

Además, el material de la unidad enfatiza: Conceptos Fundamentales Exploración con y Valor Estratégico Selenium IDE 02 04 Revisión y Optimización del Actividad Práctica: Código Automatización en Acción 05 05. • Objetivo: Validar que la automatización no es solo técnica, sino un blindaje normativo y legal para aplicaciones críticas, buscando formas que podría haber ayudado el refuerzo de la calidad y seguridad. Conceptos Fundamentales y Valor Estratégico ¿Por qué automatizar pruebas funcionales y normativas? Estas estrategias transforman código frágil en sistemas robustos y mantenibles. Instrucciones: En 2 minutos, identifiquen en sus proyectos actuales cuál de estos tres "problemas detectados" es el más recurrente: 1.Duplicación de scripts manuales. 2.Baja mantenibilidad por falta de modularidad. Exploración con Selenium IDE Introducción a la automatización con Selenium IDE ¿Qué es Selenium IDE? Selenium IDE es una herramienta poderosa y accesible que democratiza la automatización de pruebas, permitiendo a testers de todos los niveles técnicos crear scripts de prueba efectivos mediante una interfaz visual intuitiva.

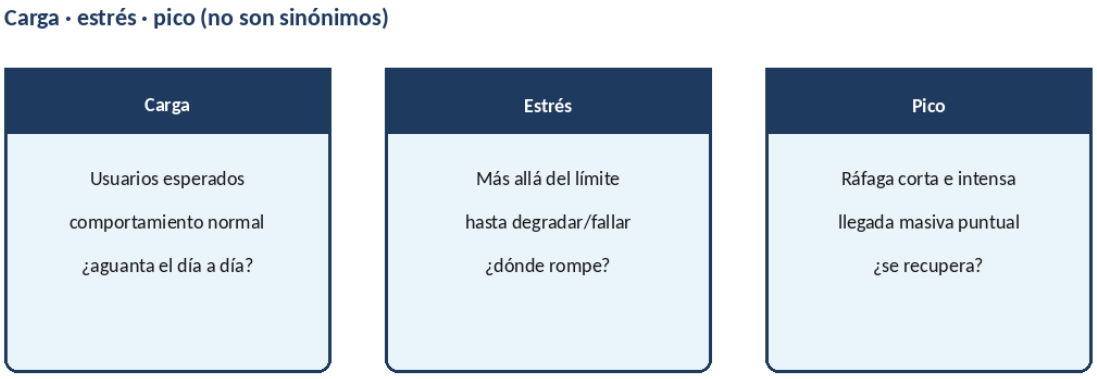
Ideas clave del bloque 3.3

- Automatizar flujos estables y de alta repetición.
- Un test *flaky* erosiona la confianza en el pipeline.
- El script también es código: se revisa y se mantiene.

3.4 Pruebas de carga, estrés y pico

3.4.1 Tres nombres que no son sinónimos

Tipo	Pregunta	Cómo se ve
Carga	¿Aguanta el uso esperado?	Usuarios concurrentes «normales»
Estrés	¿Dónde rompe?	Más allá del límite hasta degradar/fallar
Pico	¿Soporta una ráfaga y se recupera?	Subida brusca y corta



Carga, estrés y pico

3.4.2 Métricas y herramientas de estudio

Suelen observarse tiempo de respuesta, throughput (transacciones/tiempo), tasa de error, uso de CPU/memoria y el comportamiento al retirar carga (¿se recupera?). En el material del curso aparece **JMeter** (u equivalentes) para generar carga virtual y registrar resultados.

3.4.3 Cómo leer un resultado

No basta «el servidor aguantó». Hay que relacionar: perfil de carga → métricas → umbrales de negocio (p. ej. p95 de latencia) → conclusión (pasa/no pasa) → cuello de botella sospechado. Un gráfico sin hipótesis es adorno.

> **Ojo:** probar estrés en producción sin autorización y sin plan de rollback puede ser un incidente autoinfligido. Usa ambientes dedicados.

Además, el material de la unidad enfatiza: Conclusiones y Estándares Gestión de Riesgos 02 Mantenimiento del Aseguramiento Continuo de Calidad y Seguridad Ejecución de Pruebas de Estrés en un Sistema Crítico El Desafío del Límite Objetivo: Conectar la teoría con situaciones reales de alta presión. Contexto rápido: o Recordemos el caso del concierto virtual de Marc Anthony en 2021: la plataforma colapsó totalmente por no prever la carga extrema. o Las pruebas de estrés no buscan que el sistema funcione bien hoy, sino saber exactamente cuándo y cómo se romperá mañana. Introducción y Fundamentos Pausa Activa: ¿Carga o

Estrés? Tabla de Referencia: | Tipo de Prueba | Foco Principal | | Prueba de Carga | Validar requisitos bajo demanda típica o picos planificados. | Prueba de Estrés | Identificar punto de ruptura y evaluar resiliencia ante sobrecarga. Pruebas de Estrés: Evaluación Crítica de Resiliencia Las pruebas de estrés son evaluaciones sistemáticas diseñadas para determinar cómo responde un sistema ante condiciones extremas de carga, superando los límites operativos habituales. Aplicación en Sistemas Críticos Estas pruebas son fundamentales para garantizar que los sistemas mantengan su calidad, seguridad y estabilidad incluso en escenarios adversos.

Ideas clave del bloque 3.4

- Carga ≠ estrés ≠ pico.
- Define umbrales antes de ejecutar.
- Ambiente controlado y plan de detención.

3.5 Mantenimiento del aseguramiento continuo

3.5.1 Qué significa «continuo»

El aseguramiento continuo no es una campaña de una semana: es sostener calidad y seguridad mientras el producto cambia. Incluye actualizar suites automatizadas, recalibrar quality gates, reejecutar pentests cuando cambia la superficie de ataque, revisar dependencias y aprender de incidentes.

3.5.2 Ciclo práctico

1. Medir (tests, SAST/DAST, carga, incidentes).
2. Priorizar por riesgo y valor.
3. Remediar con dueño claro.
4. Retestar y dejar evidencia.
5. Ajustar el pipeline y la documentación.

3.5.3 Deuda de pruebas

Si el producto crece y la suite no, la cobertura real cae. Si la suite crece pero se vuelve frágil, el equipo la ignora (alerta fatiga). El mantenimiento equilibra **cobertura útil** y **costo de señal falsa**.

Además, el material de la unidad enfatiza: Conclusiones y Estándares Gestión de Riesgos 02 Mantenimiento del Aseguramiento Continuo de Calidad y Seguridad Ejecución de Pruebas de Estrés en un Sistema Crítico El Desafío del Límite Objetivo: Conectar la teoría con situaciones reales de alta presión. Paso 5: Análisis de resultados y mejora iterativa Identifica cuellos de botella y vulnerabilidades. Implementa mejoras técnicas y repite las pruebas para validar la efectividad de las optimizaciones aplicadas. La detección inmediata de anomalías facilita ajustes durante la prueba, optimizando el proceso de mejora continua y fortaleciendo la Indicadores principales: resiliencia del sistema. Integración con el Aseguramiento Continuo de Calidad Desarrollo Los desarrolladores escriben código y pruebas unitarias que incluyen casos de alto rendimiento. Feedback Continuo Métricas de producción informan ajustes en las pruebas de estrés para el siguiente ciclo. La integración de pruebas de estrés en el ciclo DevOps permite feedback rápido para corrección temprana. Este enfoque garantiza una mejora continua en estabilidad y seguridad, convirtiendo la resiliencia en una característica inherente del sistema, no una revisión de último momento.

Ideas clave del bloque 3.5

- Continuo = hábito con métricas y dueños, no un eslogan.
- Toda remediación cierra con retest y evidencia.
- La suite también envejece: hay que podarla y renovarla.

3.6 Chuleta de la Unidad 3

Tema	Recuerda
Pentest	Autorizado + alcance + evidencia + remediación
Black/white/grey	Según información previa
CI/CD	Integrar/probar seguido; pipeline = etapas
SAST vs DAST	Código sin ejecutar vs app corriendo
Quality gate	Umbral que puede detener el deploy
Selenium	Flujos funcionales repetibles; cuidado con flaky
Carga / estrés / pico	Esperado / más allá del límite / ráfaga
Continuo	Medir → priorizar → remediar → retest

3.7 Preguntas de autoevaluación

1. **¿Qué hace ético a un pentest?** La autorización explícita, el alcance acordado y el objetivo de mejorar la seguridad, no de dañar.
2. **¿Black box qué simula?** Un atacante externo sin información interna privilegiada.
3. **¿Un escáner de vulnerabilidades reemplaza el pentest?** No: lista hallazgos posibles; el pentest demuestra impacto en contexto.
4. **¿Qué es CI?** Integrar cambios con frecuencia disparando build y pruebas automáticas.
5. **¿SAST cuándo corre típicamente?** En etapas tempranas sobre código, sin necesidad de desplegar la app completa.
6. **¿DAST qué necesita?** Una instancia en ejecución (p. ej. staging) para atacar/observar desde fuera.
7. **¿Qué es un quality gate?** Una regla (umbrales) que impide avanzar si no se cumple.
8. **¿Por qué fallan muchos scripts Selenium?** Selectores frágiles, esperas mal hechas, datos inestables → tests *flaky*.
9. **¿Carga y estrés son lo mismo?** No: carga mira el uso esperado; estrés busca el punto de quiebre.
10. **¿Qué es una prueba de pico?** Ráfaga intensa y corta para ver degradación y recuperación.
11. **¿Qué métricas mirarías en JMeter (estudio)?** Latencia, throughput, errores, y comparación contra umbral.
12. **¿DevSecOps en una frase?** Seguridad integrada en el flujo de desarrollo y operación, no solo al final.
13. **¿Qué haces tras un hallazgo SAST crítico?** Priorizar, remediar con dueño, retest, evidencia; no solo «cerrar el ticket».
14. **¿Por qué mantener la suite?** Porque el producto cambia: sin mantenimiento, la señal se vuelve falsa o incompleta.

15. **¿Qué cierra el ciclo de aseguramiento continuo?** Retest + evidencia + ajuste de gates/docs, no solo el fix en código.

3.8 Ejercicios resueltos

3.8.1 Ejercicio 1: ¿SAST, DAST o pentest?

Tu equipo encontró (a) una inyección SQL demostrada en staging por un consultor autorizado, (b) un *code smell* de complejidad en SonarQube, (c) un header de seguridad ausente detectado por ZAP contra el ambiente QA. Clasifica.

Solución. (a) **Pentest** (explotación autorizada con impacto). (b) **SAST**/análisis estático. (c) **DAST** (app en ejecución).

3.8.2 Ejercicio 2: perfil de prueba de rendimiento

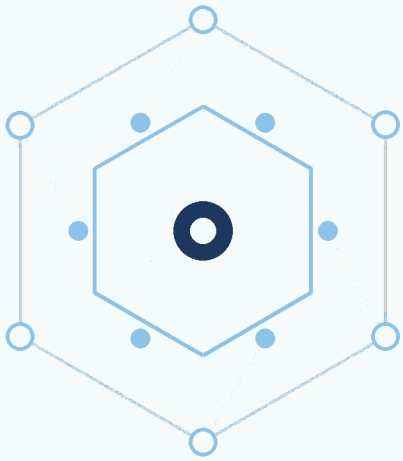
El negocio espera 200 usuarios concurrentes en horario normal, un peak de 1.000 durante 5 minutos por una campaña, y quiere saber el punto donde el error rate supera 5 %. ¿Qué pruebas diseñas?

Solución. Carga a 200 concurrentes con umbrales de latencia/error. **Pico** a 1.000 por 5 minutos midiendo degradación y recuperación. **Estrés** subiendo más allá hasta superar 5 % de error para hallar el límite. Tres perfiles, tres preguntas.

3.8.3 Ejercicio 3: quality gate

El pipeline despliega aunque haya 12 vulnerabilidades *blocker* en SAST. ¿Qué falla en el diseño del aseguramiento?

Solución. Falta (o está mal configurado) el **quality gate**: el control existe pero no detiene el avance. Corrección: definir umbral (p. ej. 0 blocker), fallar el job, notificar dueños y no promover el artefacto hasta remediar/retest.



EVALUACIÓNANX

Anexo. Glosario consolidado y

referencia rápida

Términos clave de las tres unidades, ordenados por unidad, más tablas de comparación rápida. Cada definición resume la idea esencial; el desarrollo completo está en la sección indicada.

Úsalo para repaso y para contrastar conceptos vecinos (verificación frente a validación, SAST frente a DAST, carga frente a estrés) sin releer la unidad entera.

ANX

ANEXO

Contenidos del anexo

A.1 Unidad 1 · Calidad, seguridad y cumplimiento
A.2 Unidad 2 · Diseño, ejecución y certificación
A.3 Unidad 3 · Automatización y continuo
A.4 Comparativas rápidas

Anexo. Glosario consolidado y referencia rápida

Términos de las tres unidades. Entre paréntesis, la sección donde se desarrollan.

A.1 Unidad 1 · Calidad, seguridad y cumplimiento

Término	Definición
Calidad del software	Grado en que el producto cumple requisitos y expectativas con valor real (1.1)
CIA	Confidencialidad, integridad, disponibilidad (1.1)
Security by Design	Seguridad incorporada desde el diseño (1.1)
Cumplimiento	Adherencia normativa demostrable (1.1)
ISO/IEC 25010	Modelo de calidad del producto (1.2)
ISO/IEC 27001	SGSI — gestión de seguridad de la información (1.2)
Defecto / falla / incidente	Imperfección / síntoma observable / registro formal (1.3)
Verificación / validación	¿Bien construido? / ¿Producto correcto? (1.4)
ISTQB	Marco de vocabulario y certificación de testers (1.4)
ERS	Especificación de requerimientos de software (1.4)
Shift-left	Adelantar pruebas y seguridad en el ciclo (1.4, 3.2)
Plan de pruebas	Marco estratégico de validación (1.7)
ANCI / Ley 21.663	Agencia y marco de ciberseguridad en Chile (1.6)
Ley 21.719	Protección de datos personales (1.6)

A.2 Unidad 2 · Diseño, ejecución y certificación

Término	Definición
Riesgo de prueba	Probabilidad × impacto para priorizar (2.1)
Caja negra / blanca / gris	Desde especificación / código / híbrido (2.1)
Caso de prueba	Entradas, pasos y esperado trazables a un requisito (2.3)
Partición de equivalencia	Clases de entrada de comportamiento equivalente (2.4)
BVA	Análisis de valores límite (2.4)
V(G)	Complejidad ciclomática (2.5)
Matriz de trazabilidad	Requisito ↔ casos ↔ resultado (2.6)
SonarQube / Quality Gate	SAST y condiciones de aprobación (2.7)
SAST / DAST	Análisis estático / dinámico de seguridad (2.7, 3.2)
Smoke test	Comprobación rápida de lo básico (2.8)
Informe de certificación	Dictamen con evidencias y alcance (2.9)
OWASP WSTG / Top 10	Guía de pruebas y categorías de riesgo web (2.9, 3.1)

A.3 Unidad 3 · Automatización y continuo

Término	Definición
Pentesting	Ataque simulado autorizado (3.1)
Pipeline CI/CD	Automatización build–test–deploy (3.2)

Término	Definición
DevSecOps	Seguridad integrada en DevOps (3.2)
Fail fast	Abortar ante primer fallo bloqueante (3.2)
SCA / secret scan	Dependencias vulnerables / credenciales en repo (3.2)
Selenium IDE	Automatización UI por grabación (3.3)
Prueba de carga / estrés / pico	Esperada / quiebre / ráfaga (3.4)
JMeter	Herramienta de carga y estrés (3.4)
Throughput	Peticiones u operaciones por unidad de tiempo (3.4)
MTTR	Tiempo medio de recuperación (3.2)

Términos adicionales de la Unidad 2 (repaso)

Término	Definición breve
Bloqueado	Caso que no puede ejecutarse por un defecto previo o por el entorno
BVA	*Boundary Value Analysis*, análisis de valores límite
Caja blanca	Diseño de pruebas con conocimiento del código
Caja negra	Diseño de pruebas desde entradas y salidas, sin ver el código
CI/CD	Integración y despliegue continuos
Code smell	Indicio de código difícil de mantener
Complejidad ciclomática	Número de caminos independientes de un programa, V(G)
Contenedor	Instancia en ejecución de una imagen Docker
Criterio de aceptación	Condición objetiva para aprobar un caso o cerrar un ciclo
DAST	Pruebas dinámicas de seguridad sobre la aplicación en ejecución
Deuda técnica	Esfuerzo pendiente para corregir problemas de mantenibilidad
Dictamen	Decisión final: Certificado, Certificado con reservas o No certificado
ERS	Especificación de Requerimientos de Software
Evidencia objetiva	Registro verificable, trazable y suficiente que respalda un hallazgo
Hotspot de seguridad	Código sensible que requiere revisión humana
Imagen Docker	Plantilla inmutable con todo lo necesario para ejecutar una aplicación
No conformidad	Incumplimiento de un requisito (mayor o menor)
OWASP	Fundación que publica guías abiertas de seguridad de aplicaciones (Top 10, WSTG)
Prioridad	Urgencia de corrección según el negocio
Quality Gate	Conjunto de condiciones que el código debe cumplir para aprobarse en SonarQube
Regresión	Reaparición de un defecto, o aparición de uno nuevo, después de un cambio
SAST	Análisis estático de seguridad del código fuente
Severidad	Impacto técnico de un defecto

Término	Definición breve
Shift-left	Adelantar las pruebas a las primeras etapas del ciclo de vida
Smoke test	Prueba rápida de funciones básicas antes de ejecutar el resto
Trazabilidad	Relación verificable entre requisitos, casos, resultados e incidentes
Walkthrough	Revisión guiada por el autor del código o documento
WSTG	*Web Security Testing Guide* de OWASP

A.4 Comparativas rápidas

Calidad del producto frente a SGSI

Aspecto	ISO/IEC 25010	ISO/IEC 27001
Objeto	Producto software	Sistema de gestión de seguridad de la información
Pregunta	¿Qué tan «bueno» es el producto en sus atributos?	¿Están gestionados los riesgos de información?
Uso en pruebas	Orientar tipos y NFRs	Orientar controles y auditoría de seguridad

SAST frente a DAST

Aspecto	SAST	DAST
Entrada	Código fuente	App en ejecución
Momento	Muy temprano	Con entorno desplegado
Hallazgo	Ubicación en código	Explotabilidad externa
Ejemplo	SonarQube	OWASP ZAP

Carga frente a estrés

Aspecto	Carga	Estrés
Intensidad	Dentro o en el pico esperado	Por encima del límite
Pregunta	¿Cumple tiempos bajo demanda prevista?	¿Dónde y cómo se rompe? ¿Se recupera?
Uso	Validar NFR de desempeño	Capacidad y resiliencia

Manual frente a automatizado en el pipeline

Aspecto	Manual	Automatizado
Fortaleza	Exploratorio, usabilidad, juicio	Regresión, velocidad, consistencia
Debilidad	Lento, variable	Costo de mantenimiento de scripts
Encaje	Sesiones exploratorias, UAT	CI en cada commit / nightly

Unidad 1 → 2 → 3 (hilo conductor)

Unidad	Entrega mental
1	Vocabulario, normas, plan
2	Casos, evidencias, dictamen
3	Automatizar y sostener en el tiempo

Fin de la guía unificada. Contrasta siempre umbrales, versiones de herramientas y artículos legales con la fuente vigente indicada en tu material de clase.

Fin de la guía unificada — ISY1102 Seguridad y calidad en el desarrollo de software.